Rust

K19G

2026-10-11

Table of contents

Rust **19**
A Complete Guide to Safe, Concurrent, High-Performance Programming 19
Introduction: Why Rust Matters 19
The Problem Rust Solves 19
How Rust Achieves This 20
What You Will Learn 20
How to Read This Book 20
The Current State of Rust 20
What Rust Is Not 21
Formats 21

Getting Started with Rust **22**

The Story of Rust: History and Philosophy **23**

The Story of Rust: History and Philosophy **24**
Core Philosophy 24

Installing Rust with rustup **25**

Installing Rust with rustup **26**
Installing rustup 26
Managing Toolchains 26

Your First Program: Hello World **27**

Your First Program: Hello World **28**
Direct Compilation with rustc 28
Deconstructing the Program 28

Understanding rustc and cargo **29**

Understanding rustc and cargo **30**
The Role of Cargo 30
Common Cargo Commands 30

Exploring a Cargo Project Structure **31**

Exploring a Cargo Project Structure **32**
Standard Directory Layout 32
The Cargo.toml Manifest 32

Summary 33

Summary 34
Chapter Key Takeaways 34

Language Basics: Variables, Types, and Expressions 35

Variables, Mutability, and Shadowing 36

Variables, Mutability, and Shadowing 37
Mutability with `mut` 37
Variable Shadowing 37

Primitive Data Types in Detail 38

Primitive Data Types in Detail 39
Scalar Types 39

Compound Types: Tuples and Arrays 40

Compound Types: Tuples and Arrays 41
Tuples 41
Arrays 41

Functions, Statements, and Expressions 42

Functions, Statements, and Expressions 43
Statements vs. Expressions 43

Control Flow: if, loop, while, and for 44

Control Flow: if, loop, while, and for 45
`if` as an Expression 45
Loops: `loop`, `while`, and `for` 45

Summary 46

Summary 47
Chapter Key Takeaways 47

Ownership: The Heart of Rust 48

What is Ownership and Why Does It Exist? 49

What is Ownership and Why Does It Exist? 50
The Three Rules of Ownership 50

Stack vs Heap Memory in Rust 51

Stack vs Heap Memory in Rust **52**
The Stack . . . 52
The Heap . . . 52

Move Semantics Explained **53**

Move Semantics Explained **54**
Why Rust Moves Instead of Shallow Copying . . . 54

Copy Types and Clone **55**

Copy Types and Clone **56**
The `Copy` Trait . . . 56
The `Clone` Trait . . . 56

The Drop Trait and Resource Cleanup **57**

The Drop Trait and Resource Cleanup **58**
Custom Drop Implementations . . . 58

Summary **59**

Summary **60**
Chapter Key Takeaways . . . 60

Borrowing, References, and Slices **61**

Shared and Mutable References **62**

Shared and Mutable References **63**
Shared References (`&T`) . . . 63
Mutable References (`&mut T`) . . . 63

The Borrowing Rules Demystified **64**

The Borrowing Rules Demystified **65**
Why the Rules Exist . . . 65

Dangling References Prevented at Compile Time **66**

Dangling References Prevented at Compile Time **67**

String Slices and &str **68**

String Slices and &str **69**
`String` vs `&str` . . . 69

Slice Types and Range Syntax **70**

Slice Types and Range Syntax **71**

Summary 72

Summary 73
Chapter Key Takeaways 73

Structs, Enums, and Pattern Matching 74

Defining and Using Structs 75

Defining and Using Structs 76

Enumerations Beyond Simple Tags 77

Enumerations Beyond Simple Tags 78

The match Expression in Depth 79

The match Expression in Depth 80

if let and while let for Partial Matching 81

if let and while let for Partial Matching 82

Real-World Data Modeling with Enums 83

Real-World Data Modeling with Enums 84

Summary 85

Summary 86
Chapter Key Takeaways 86

Modules, Crates, and Visibility 87

Modules and the Filesystem 88

Modules and the Filesystem 89
The Modern Module Mapping 89

The use Statement and Path Resolution 90

The use Statement and Path Resolution 91

Public vs Private Visibility Rules 92

Public vs Private Visibility Rules 93

Crates, Packages, and Cargo Workspaces 94

Crates, Packages, and Cargo Workspaces 95

Re-exporting and Organizing Large Projects **96**

Re-exporting and Organizing Large Projects **97**

Summary **98**

Summary **99**
Chapter Key Takeaways 99

Collections and Strings **100**

Vec and Dynamic Arrays **101**

Vec and Dynamic Arrays **102**

String vs &str Deep Dive **103**

String vs &str Deep Dive **104**

HashMap and BTreeMap for Key-Value Storage **105**

HashMap and BTreeMap for Key-Value Storage **106**

Other Collections: HashSet, BinaryHeap, LinkedList **107**

Other Collections: HashSet, BinaryHeap, LinkedList **108**

Efficient Collection Operations **109**

Efficient Collection Operations **110**

Summary **111**

Summary **112**
Chapter Key Takeaways 112

Project 1: Building a CLI Task Manager **113**

Building a CLI Task Manager **114**

Building a CLI Task Manager **115**

Testing the Task Manager **117**

Testing the Task Manager **118**

Benchmarking the Task Manager **119**

Benchmarking the Task Manager **120**

Deploying the Task Manager **121**

Deploying the Task Manager **122**

Error Handling in Rust **123**

Unrecoverable Errors with panic! **124**

Unrecoverable Errors with panic! **125**

Recoverable Errors with Result **126**

Recoverable Errors with Result **127**

The Option Type for Absent Values **128**

The Option Type for Absent Values **129**

Propagating Errors with the ? Operator **130**

Propagating Errors with the ? Operator **131**

Custom Error Types and thiserror **132**

Custom Error Types and thiserror **133**

The Never Type `!` and Infallible **134**

The Never Type `!` and `Infallible` **135**
- Thesis . . . 135
- Mental model . . . 135
- Worked examples . . . 136
  - Case 1: `!` as a type — cannot-fail results, irrefutable `Ok` . . . 136
  - Case 2: the fallback change — code that relied on `()` by accident . . . 138
  - Case 3: the panicking closure . . . 140
  - Case 4: `Infallible` and `!` are one type now . . . 141
- Where `!` belongs in error handling . . . 142
- The trap . . . 142
- The boring rule . . . 143
- Try this . . . 143
- Sources . . . 143

Summary **145**

Summary **146**
- Chapter Key Takeaways . . . 146

Generics, Traits, and Associated Types 147

Generic Functions and Data Structures 148

Generic Functions and Data Structures 149

Defining and Implementing Traits 150

Defining and Implementing Traits 151

Trait Bounds and Where Clauses 152

Trait Bounds and Where Clauses 153

Associated Types vs Generics 154

Associated Types vs Generics 155

Trait Objects and Dynamic Dispatch 156

Trait Objects and Dynamic Dispatch 157

Generic Const Arguments (GCA) 158

Generic Const Arguments (GCA) 159
- Thesis . . . 159
- Mental model . . . 159
- Worked examples . . . 160
  - Case 0: stable still rejects `N + 1` . . . 160
  - Case 1: ADT const arguments with `gca_adts` . . . 161
  - Case 2: Associated constants with `gca_const_items` . . . 162
  - Case 3: Macroless (experimental) . . . 162
  - Case 4: What still hurts — arithmetic inside `gca!` . . . 163
- The trap . . . 164
- The boring rule . . . 164
- Try this . . . 164
- Sources . . . 164

Summary 165

Summary 166
- Chapter Key Takeaways . . . 166

Lifetimes: Complete Understanding 167

What Lifetimes Are (and Are Not) 168

What Lifetimes Are (and Are Not) 169
- What Lifetimes Are Not . . . 169

The Lifetime Elision Rules **170**

The Lifetime Elision Rules **171**

Annotating Lifetimes in Functions and Structs **172**

Annotating Lifetimes in Functions and Structs **173**

Lifetime Bounds on Traits **174**

Lifetime Bounds on Traits **175**

Advanced Lifetime Patterns **176**

Advanced Lifetime Patterns **177**
The `'static` Lifetime 177

Summary **178**

Summary **179**
Chapter Key Takeaways 179

Smart Pointers and Interior Mutability **180**

Box for Heap Allocation **181**

Box for Heap Allocation **182**

Reference Counting with Rc and Arc **183**

Reference Counting with Rc and Arc **184**

Interior Mutability with RefCell **185**

Interior Mutability with RefCell **186**

Cell and Mutex for Shared State **187**

Cell and Mutex for Shared State **188**

Weak References and Breaking Cycles **189**

Weak References and Breaking Cycles **190**

LazyLock and OnceLock **191**

LazyLock and OnceLock **192**
Mental model 192
Worked examples 193
Case 1: `LazyLock` for a process-wide regex 193
Case 2: `OnceLock` when init needs an argument 193

Case 3: Fallible init without poisoning `LazyLock` . . . 194
Case 4: Single-threaded `LazyCell` . . . 195
Choosing among the options . . . 195
Pitfalls . . . 195
Try this . . . 196
Sources . . . 196

MaybeUninit and Uninitialized Memory **197**

MaybeUninit and Uninitialized Memory **198**
Mental model . . . 198
Worked examples . . . 199
Case 1: `write` then `assume_init` . . . 199
Case 2: Out-pointer style . . . 199
Case 3: Array element-by-element . . . 199
Case 4: Field-by-field struct init . . . 200
Case 5: `zeroed` pitfalls . . . 201
Choosing the API . . . 201
Pitfalls . . . 202
Try this . . . 202
Sources . . . 202

Custom Allocators and the Allocator API **203**

Custom Allocators and the Allocator API **204**
Thesis . . . 204
Mental model . . . 204
Worked examples . . . 205
Case 1: a counting allocator and a bump arena . . . 205
Case 2: what the allocator parameter costs . . . 209
Case 3: an allocator says no . . . 210
Case 4: what is *not* stable yet . . . 212
The trap . . . 214
A collection outlives its arena . . . 214
`Vec<T, A>` does not fit where `Vec<T>` is expected . . . 215
Implementing the trait is a safety contract . . . 216
Choosing an approach . . . 216
The boring rule . . . 216
Try this . . . 217
Sources . . . 217

Summary **218**

Summary **219**
Chapter Key Takeaways . . . 219

Iterators, Closures, and Functional Patterns 220

The Iterator Trait and Protocol 221

The Iterator Trait and Protocol 222

Creating Custom Iterators 223

Creating Custom Iterators 224

Closures and Capture Semantics 225

Closures and Capture Semantics 226

Higher-Order Functions and Combinators 227

Higher-Order Functions and Combinators 228

Lazy Evaluation and Performance 229

Lazy Evaluation and Performance 230

Summary 231

Summary 232

Chapter Key Takeaways 232

Concurrency: Fearless Parallelism 233

Spawning Threads with std::thread 234

Spawning Threads with std::thread 235

Message Passing with Channels 236

Message Passing with Channels 237

Shared State with Mutex and RwLock 238

Shared State with Mutex and RwLock 239

Atomic Operations for Lock-Free Code 240

Atomic Operations for Lock-Free Code 241

Send and Sync Traits Explained 242

Send and Sync Traits Explained 243

Summary 244

Summary **245**
Chapter Key Takeaways 245

Project 3: Parallel Data Processing Pipeline with rayon **246**

Parallel Data Processing Pipeline with rayon **247**

Parallel Data Processing Pipeline with rayon **248**

Testing the Text Analyzer **249**

Testing the Text Analyzer **250**

Benchmarking the Text Analyzer **251**

Benchmarking the Text Analyzer **252**

Deploying the Text Analyzer **253**

Deploying the Text Analyzer **254**

Asynchronous Programming with async/await **255**

The Future Trait and Async Basics **256**

The Future Trait and Async Basics **257**

Running Async Code with Tokio **258**

Running Async Code with Tokio **259**

Pinning and Self-Referential Types **260**

Pinning and Self-Referential Types **261**

Async Streams and Error Handling **262**

Async Streams and Error Handling **263**

Building Production Async Applications **264**

Building Production Async Applications **265**

Tokio and Rayon: Separate Lanes for I/O and CPU Work **266**

Tokio and Rayon: Separate Lanes for I/O and CPU Work **267**
Thesis 267
Mental model 267
Worked examples 268
Case 1: Same throughput, very different latency 268

Case 2: Under a CPU limit . . . 270
Case 3: Bound the work in flight . . . 271
The trap . . . 271
The boring rule . . . 272
Try this . . . 273
Sources . . . 273

Summary **274**

Summary **275**
Chapter Key Takeaways . . . 275

Macros, Unsafe Rust, and FFI **276**

Declarative Macros with macro_rules! **277**

Declarative Macros with macro_rules! **278**

Procedural Macros and Derive **279**

Procedural Macros and Derive **280**

Understanding unsafe Rust **281**

Understanding unsafe Rust **282**

Memory Layout and Representation **283**

Memory Layout and Representation **284**

Foreign Function Interfaces **285**

Foreign Function Interfaces **286**

C-Variadic Functions and VaList **287**

C-Variadic Functions and `VaList` **288**
Thesis . . . 288
Mental model . . . 288
Worked examples . . . 289
Case 1: count-prefixed list, defined and called in Rust . . . 289
Case 2: a printf-style logger implemented in Rust, called from C . . . 290
Case 3: the header catches caller mistakes before they become UB . . . 293
Case 4: what the compiler refuses . . . 294
The trap . . . 296
The boring rule . . . 297
Try this . . . 297
Sources . . . 298

Summary **299**

Summary **300**
Chapter Key Takeaways 300

Testing, Documentation, Performance, and Deployment **301**

Unit Tests, Integration Tests, and Doctests **302**

Unit Tests, Integration Tests, and Doctests **303**
Unit Tests 303
Integration Tests 303
Documentation Tests (Doctests) 303

Benchmarking with Criterion **304**

Benchmarking with Criterion **305**

Writing Great Documentation **306**

Writing Great Documentation **307**

Profiling and Optimization Techniques **308**

Profiling and Optimization Techniques **309**

Building and Deploying Rust Applications **310**

Building and Deploying Rust Applications **311**

Summary **312**

Summary **313**
Chapter Key Takeaways 313

The Rust Ecosystem: Essential Crates for Production Development **314**

serde: Serialization and Deserialization **315**

serde: Serialization and Deserialization **316**

clap: Command-Line Argument Parsing **317**

clap: Command-Line Argument Parsing **318**

reqwest: HTTP Client **319**

reqwest: HTTP Client **320**

axum: Web Framework **321**

axum: Web Framework **322**

actix-web: High-Performance Web Framework **323**

actix-web: High-Performance Web Framework **324**

Project 2: Async HTTP API with axum, sqlx, and tracing **325**

Project 2: Async HTTP API with axum, sqlx, and tracing **326**

Testing the User API **327**

Testing the User API **328**

Benchmarking the User API **329**

Benchmarking the User API **330**

Deploying the User API **331**

Deploying the User API **332**

sqlx: Async Database Access **333**

sqlx: Async Database Access **334**

rayon: Data Parallelism **335**

rayon: Data Parallelism **336**

tracing: Structured Logging and Observability **337**

tracing: Structured Logging and Observability **338**

chrono: Date and Time **339**

chrono: Date and Time **340**

uuid: Universally Unique Identifiers **341**

uuid: Universally Unique Identifiers **342**

rand: Random Number Generation **343**

rand: Random Number Generation **344**

regex: Regular Expressions **345**

regex: Regular Expressions **346**

bytes: Efficient Binary Buffers **347**

bytes: Efficient Binary Buffers **348**

futures: Async Primitives **349**

futures: Async Primitives **350**

Summary **351**

Summary **352**
Chapter Key Takeaways 352

Advanced Topics: Compiler, Debugging, Security, and Anti-Patterns **353**

The Rust Compilation Pipeline **354**

The Rust Compilation Pipeline **355**

Debugging Rust Programs **356**

Debugging Rust Programs **357**

Security Considerations in Rust **358**

Security Considerations in Rust **359**

Common Anti-Patterns and How to Avoid Them **360**

Common Anti-Patterns and How to Avoid Them **361**

Summary **362**

Summary **363**
Chapter Key Takeaways 363

WebAssembly with Rust **364**

Why Rust for WebAssembly? **365**

Why Rust for WebAssembly? **366**

WebAssembly Fundamentals **367**

WebAssembly Fundamentals **368**

Setting Up: wasm-pack and wasm-bindgen **369**

Setting Up: wasm-pack and wasm-bindgen **370**

Building Your First WebAssembly Module **371**

Building Your First WebAssembly Module **372**

Interacting with the DOM **373**

Interacting with the DOM **374**

Handling Events and Async **375**

Handling Events and Async **376**

Performance Considerations **377**

Performance Considerations **378**

Server-Side WebAssembly **379**

Server-Side WebAssembly **380**

Publishing and Distribution **381**

Publishing and Distribution **382**

Summary **383**

Summary **384**
Chapter Key Takeaways 384

Embedded Rust and no_std Development **385**

Understanding no_std **386**

Understanding no_std **387**

Choosing a Target: Cortex-M **388**

Choosing a Target: Cortex-M **389**

Writing an Embedded Application: Blinking an LED **390**

Writing an Embedded Application: Blinking an LED **391**

Memory Management Without a Heap **392**

Memory Management Without a Heap **393**

Interrupts and Concurrency **394**

Interrupts and Concurrency **395**

Choosing an Executor or RTOS **396**

Choosing an Executor or RTOS **397**

Testing Embedded Code **398**

Testing Embedded Code **399**

Toolchain and Debugging **400**

Toolchain and Debugging **401**

Summary **402**

Summary **403**
Chapter Key Takeaways 403

Conclusion: The Rust Mindset **404**

Core Principles **405**

Core Principles **406**

Where Rust Shines **407**

Where Rust Shines **408**

Where Rust May Not Be the Best Fit **409**

Where Rust May Not Be the Best Fit **410**

Continuing Your Journey **411**

Continuing Your Journey **412**

Final Thoughts **413**

Final Thoughts **414**

Rust

A Complete Guide to Safe, Concurrent, High-Performance Programming

A systematic, production-grade guide to writing Rust the way it lasts in real systems: **safe, concurrent, and high-performance**. Rust eliminates entire categories of critical bugs—null pointer dereferences, data races, buffer overflows, and use-after-free vulnerabilities—without the runtime overhead of a garbage collector.

This book provides a direct, comprehensive path from first principles to advanced production systems.

Introduction: Why Rust Matters

For decades, systems programming forced engineers into an uncomfortable compromise:

1. **Control and Speed without Safety:** Languages like C and C++ provide bare-metal hardware access, deterministic resource destruction, and peak performance, but leave memory safety, thread safety, and undefined behavior to developer discipline. Decades of CVE audits prove that even the most disciplined teams cannot prevent memory corruption at scale.
2. **Safety without Bare-Metal Control:** Managed languages like Java, Go, Python, and C# enforce memory safety through automatic garbage collection and runtime checks. However, garbage collection introduces non-deterministic pause times, memory footprint overhead, and indirection that make them less suitable for low-latency systems, kernel drivers, game engines, and embedded microcontrollers.

Rust breaks this compromise by achieving **compile-time memory safety without a garbage collector**.

The Problem Rust Solves

Software reliability and security crises predominantly stem from memory safety bugs and concurrency defects: - **Spatial memory safety violations:** Buffer overruns, out-of-bounds array reads and writes. - **Temporal memory safety violations:** Use-after-free, double free, dangling pointers. - **Concurrency bugs:** Data races occurring when two threads access the same memory location concurrently with at least one writer and no synchronization.

Rust guarantees that properly written safe code cannot experience any of these defects. If your code compiles, these failure modes are mathematically disproven by the compiler's borrow checker.

How Rust Achieves This

Rust replaces runtime garbage collection and manual memory management with three core mechanisms: 1. **Ownership:** Every value has a single owner bound to lexical scope. When the owner goes out of scope, the memory is deterministically freed (`Drop`). 2. **Borrowing & References:** References allow inspection or mutation of data without taking ownership. Rust enforces aliasing XOR mutability at compile time: you may have either any number of immutable references (`&T`) *or* exactly one mutable reference (`&mut T`), but never both simultaneously. 3. **Lifetimes:** The compiler tracks the validity of every reference, preventing dangling pointers across function boundaries and data structures at compile time without runtime overhead.

What You Will Learn

Across 20 comprehensive chapters and three end-to-end production capstones, this book covers: - Core language syntax, primitive/compound types, and zero-cost control flow. - Deep mechanical mastery of ownership, stack vs. heap allocation, move semantics, and lifetimes. - Robust data modeling with algebraic data types (`struct`, `enum`) and expressive pattern matching. - Explicit, resilient error handling with `Result`, `Option`, and the `?` propagation operator. - Parametric polymorphism with generics, trait bounds, and dynamic dispatch via trait objects. - Fearless concurrency using threads, message-passing channels, and atomic primitives. - Asynchronous programming with `async`/`await`, `Future`, and the Tokio runtime. - Systems-level metaprogramming with declarative macros, procedural macros, and foreign function interfaces (FFI). - Production engineering: profiling, benchmarking with Criterion, Criterion, cargo packaging, WebAssembly (`wasm-pack`), and embedded `no_std` firmware.

How to Read This Book

1. **Linear Progression:** Chapters 1 through 13 form the core foundation. Read them sequentially; each chapter relies strictly on concepts verified in preceding sections.
2. **Complete, Runnable Code:** Every listing is self-contained. Copy the code into your editor, compile it with `cargo check` and `cargo run`, and observe compiler messages.
3. **Fight and Befriend the Borrow Checker:** When the compiler rejects your code, read the compiler diagnostic carefully. Rust diagnostics are world-class teaching tools explaining what invariant was violated and how to rectify it.
4. **Capstones & Projects:** Work through the three integrated projects (CLI Task Manager, Parallel Data Pipeline with Rayon, and Async HTTP API with Axum and SQLx) to solidify architecture and integration patterns.

The Current State of Rust

Rust is mature, battle-tested, and officially adopted across industry leaders for core infrastructure: - The Linux kernel has official support for Rust drivers and kernel modules. - Major cloud hypervisors and networking layers (AWS Firecracker, Cloudflare Workers, Android OS) are authored in Rust. - Standards organizations and government agencies (CISA, NSA) recommend transitioning memory-unsafe codebases to memory-safe languages like Rust.

What Rust Is Not

To set clear expectations, Rust is not: - **An object-oriented language with class hierarchies:** Rust uses composition, traits, and algebraic data types rather than inheritance. - **A rapid prototyping scripting language:** Rust demands explicit thinking up front about ownership, error conditions, and data lifetimes. - **Magic:** Zero-cost abstractions mean you don't pay runtime costs for high-level ergonomics; however, you must understand machine layouts and memory representation.

Formats

This book is available in multiple formats via the library portal: - **HTML** — responsive layout with interactive syntax highlighting and dual light/dark themes. - **PDF** — typeset document suitable for offline reading and archiving. - **EPUB** — standardized eBook format for mobile devices and e-readers.

Getting Started with Rust

The Story of Rust: History and Philosophy

The Story of Rust: History and Philosophy

Rust began in 2006 as a personal project by Graydon Hoare, a language engineer at Mozilla. At the time, Mozilla was grappling with the massive complexity and vulnerability surface of the Gecko rendering engine in Firefox, written in C++. In multi-threaded C++, memory corruption vulnerabilities (such as use-after-free and buffer overflows) and concurrency bugs (data races) were extraordinarily difficult to prevent, even for seasoned systems developers.

Mozilla sponsored Rust in 2009 with a distinct mission: **create a modern systems language that guarantees memory safety and thread safety without sacrificing bare-metal speed or requiring a garbage collector**.

Core Philosophy

The design of Rust rests on three non-negotiable pillars: 1. **Safety:** Eliminate undefined behavior, dangling pointers, null dereferences, and data races at compile time. 2. **Speed:** Provide zero-cost abstractions—constructs that compile down to assembly indistinguishable from hand-written C. 3. **Concurrency:** "Fearless concurrency"—the compiler prevents data races across threads using the exact same ownership model used for memory safety.

```
// A safe, deterministic greeting
fn main() {
    let project = "Rust";
    let year = 2026;
    println!("Welcome to {}, built for safety and speed in {}.", project, year);
}
```

Installing Rust with rustup

Installing Rust with rustup

The primary and official method for installing and managing Rust toolchains across all platforms is `rustup`.

Installing rustup

On Linux and macOS, install `rustup` via your terminal:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Follow the default prompts. This installs: - `rustc`: The Rust compiler. - `cargo`: The package manager and build system. - `rustup`: The toolchain multiplexer. - `rust-std`: Standard library binaries for your host architecture.

Managing Toolchains

Configure your shell environment:

```
source "$HOME/.cargo/env"
rustc --version
cargo --version
```

To update your toolchain to the latest stable release:

```
rustup update stable
```

Your First Program: Hello World

Your First Program: Hello World

Let's begin with the canonical starting point: a minimal standalone Rust source file.

Direct Compilation with rustc

Create a file named `main.rs`:

```
// main.rs
fn main() {
    println!("Hello, world!");
}
```

Compile and run directly using `rustc`:

```
rustc main.rs
./main
```

Output:

```
Hello, world!
```

Deconstructing the Program

- `fn main() { ... }`: The `main` function is the entry point of every executable Rust program. It takes no arguments and returns unit `()`.
- `println!(...)`: In Rust, names ending in an exclamation mark `!` invoke **macros** rather than standard functions. `println!` parses format strings at compile time to ensure type safety.

Understanding rustc and cargo

Understanding rustc and cargo

While `rustc` is the underlying compiler, production Rust projects are almost exclusively managed with **Cargo**.

The Role of Cargo

Cargo handles four essential engineering responsibilities: 1. **Dependency Resolution:** Downloading, pinning, and compiling external crates from crates.io. 2. **Build Orchestration:** Invoking `rustc` with optimal optimization flags, link arguments, and target settings. 3. **Test & Benchmark Execution:** Discovering and executing unit tests, integration tests, and performance benchmarks. 4. **Documentation Generation:** Generating standard HTML documentation directly from doc-comments.

Common Cargo Commands

```
cargo check   # Fast syntax and type verification without binary emission
cargo build   # Compile debug build with symbols into target/debug
cargo run     # Build and execute the binary
cargo test    # Run automated test harness
cargo doc     # Build HTML documentation for current project and dependencies
```

Exploring a Cargo Project Structure

Exploring a Cargo Project Structure

Initialize a new project with Cargo:

```
cargo new desk_service --bin
cd desk_service
```

Standard Directory Layout

```
desk_service/
   Cargo.lock       # Exact resolved dependency graph
   Cargo.toml       # Manifest file: project metadata & dependencies
   src/
       main.rs      # Binary root entry point
   target/          # Build artifacts and compilation cache (gitignored)
```

The Cargo.toml Manifest

```
[package]
name = "desk_service"
version = "0.1.0"
edition = "2024"

[dependencies]
```

The `edition = "2024"` flag configures the language edition. Rust guarantees backwards compatibility across editions through epoch-based stability.

Summary

Summary

Chapter Key Takeaways

- Rust provides zero-cost memory safety without a garbage collector through static ownership analysis.
- `rustup` is the toolchain manager; keep it updated with `rustup update`.
- `rustc` compiles code, but day-to-day workflow centers on `cargo check`, `cargo build`, `cargo test`, and `cargo run`.
- Every Rust binary begins at `fn main()` and projects are configured through `Cargo.toml`.

Language Basics: Variables, Types, and Expressions

Variables, Mutability, and Shadowing

Variables, Mutability, and Shadowing

By default, variables in Rust are **immutable**. Once bound to a value, the binding cannot be modified.

Mutability with `mut`

To allow mutation, explicitly declare the binding with `mut`:

```
fn main() {
    let mut counter = 10;
    println!("Initial counter: {}", counter);
    counter += 5;
    println!("Updated counter: {}", counter);
}
```

Variable Shadowing

Rust allows redeclaring a variable with the same name in the same scope using `let`. This is **shadowing**:

```
fn main() {
    let spaces = "   ";
    let spaces = spaces.len(); // Shadows spaces: changes type from &str to usize
    println!("Number of spaces: {}", spaces);
}
```

Shadowing creates a new binding, allowing type transformation without requiring distinct variable names.

Primitive Data Types in Detail

Primitive Data Types in Detail

Rust is statically and strongly typed with full type inference.

Scalar Types

1. **Integers:**
   - Signed: `i8`, `i16`, `i32`, `i64`, `i128`, `isize` (pointer width)
   - Unsigned: `u8`, `u16`, `u32`, `u64`, `u128`, `usize` (pointer width, indexing)
2. **Floating-point:** `f32`, `f64` (IEEE 754-2008)
3. **Boolean:** `bool` (`true` or `false`, 1 byte representation)
4. **Character:** `char` (4-byte Unicode scalar value)

```
fn main() {
    let count: usize = 42;
    let temperature: f64 = 98.6;
    let is_active: bool = true;
    let emoji: char = ' ';

    println!("Scalar values: {}, {}, {}, {}", count, temperature, is_active, emoji);
}
```

Compound Types: Tuples and Arrays

Compound Types: Tuples and Arrays

Compound types group multiple values into a single type.

Tuples

A tuple has a fixed length and can contain elements of different types:

```
fn main() {
    let point: (i32, f64, &str) = (10, 3.14, "origin");
    let (x, y, label) = point; // Destructuring
    println!("Point {}: x = {}, y = {}", label, x, y);
    println!("Direct access: first = {}", point.0);
}
```

Arrays

An array has a fixed length, is stack-allocated, and elements must be of the **same type**:

```
fn main() {
    let buffer: [u8; 4] = [0xDE, 0xAD, 0xBE, 0xEF];
    println!("Array length: {}, first: 0x{:X}", buffer.len(), buffer[0]);

    let initialized = [0u32; 16]; // 16 elements initialized to 0
    println!("Initialized array length: {}", initialized.len());
}
```

Functions, Statements, and Expressions

Functions, Statements, and Expressions

Rust is an expression-oriented language. Almost every construct evaluates to a value.

Statements vs. Expressions

- **Statements:** Instructions that perform an action and do not return a value. Ending an expression with a semicolon `;` turns it into a statement returning unit `()`.
- **Expressions:** Evaluate to a resulting value. Omission of the final semicolon returns the expression value.

```
fn add(a: i32, b: i32) -> i32 {
    a + b // No semicolon: implicit return
}

fn compute_block(factor: i32) -> i32 {
    let result = {
        let base = 100;
        base * factor // Expression evaluated and bound to result
    };
    result
}

fn main() {
    println!("Add: {}", add(10, 20));
    println!("Compute block: {}", compute_block(3));
}
```

Control Flow: if, loop, while, and for

Control Flow: if, loop, while, and for

Control flow constructs in Rust are expressions that can return values.

`if` as an Expression

```
fn main() {
    let condition = true;
    let number = if condition { 5 } else { 6 };
    println!("Number: {}", number);
}
```

Loops: `loop`, `while`, and `for`

```
fn main() {
    // 1. loop with value return
    let mut counter = 0;
    let result = loop {
        counter += 1;
        if counter == 10 {
            break counter * 2;
        }
    };
    println!("Loop result: {}", result);

    // 2. while loop
    let mut n = 3;
    while n > 0 {
        println!("{}!", n);
        n -= 1;
    }

    // 3. for loop over iterators
    for item in 0..3 {
        println!("Index: {}", item);
    }
}
```

Summary

Summary

Chapter Key Takeaways

- Variables are immutable by default; use `mut` to allow change.
- Shadowing (`let x = ...`) rebinds names and enables type changes.
- Rust primitives include scalar types (integers, floats, booleans, chars) and compound types (tuples, fixed-size arrays).
- Rust is expression-oriented: blocks and control flow evaluate directly to values.

Ownership: The Heart of Rust

What is Ownership and Why Does It Exist?

What is Ownership and Why Does It Exist?

Ownership is Rust's central memory management innovation. It governs how heap memory is allocated, accessed, and reclaimed without a garbage collector or manual `free()` calls.

The Three Rules of Ownership

1. **Each value in Rust has an owner.**
2. **There can only be one owner at a time.**
3. **When the owner goes out of scope, the value is dropped.**

```
fn main() {
    {
        let s = String::from("desk"); // s is valid from this point forward
        println!("Inside scope: {}", s);
    } // s goes out of scope; String memory is automatically freed here
}
```

Stack vs Heap Memory in Rust

Stack vs Heap Memory in Rust

Understanding where data lives is essential for writing efficient Rust code.

The Stack

- Fast, LIFO (Last In, First Out) allocation.
- All data stored on the stack must have a known, fixed size at compile time.
- Primitive types, fixed arrays, and function frames live on the stack.

The Heap

- Dynamic memory allocated at runtime.
- The operating system locates a contiguous region of memory, marks it in use, and returns a pointer.
- Slower than stack allocation; requires dereferencing pointers.
- Dynamically sized types like `String` and `Vec<T>` store metadata on the stack (pointer, length, capacity) pointing to heap buffers.

Move Semantics Explained

Move Semantics Explained

When a heap-backed value is assigned to another binding, passed to a function, or returned from a function, ownership is **moved**.

```
fn main() {
    let s1 = String::from("orders");
    let s2 = s1; // s1 is MOVED to s2

    // println!("{}", s1); // COMPILE ERROR: value borrowed here after move
    println!("s2 owns the heap buffer: {}", s2);
}
```

Why Rust Moves Instead of Shallow Copying

If `s1` and `s2` both owned the same heap pointer, when both went out of scope, they would attempt to free the same memory twice—a catastrophic **double free** bug. Rust prevents this by invalidating `s1` at compile time with zero runtime cost.

Copy Types and Clone

Copy Types and Clone

The `Copy` Trait

Types that live entirely on the stack and have fixed sizes implement the `Copy` trait. Assignment of `Copy` types performs a bitwise copy rather than a move:

```
fn main() {
    let x: i32 = 42;
    let y = x; // Bitwise copy
    println!("x = {}, y = {}", x, y); // Both valid!
}
```

The `Clone` Trait

For heap-allocated types where an explicit, potentially expensive deep copy is desired, use `.clone()`:

```
fn main() {
    let s1 = String::from("records");
    let s2 = s1.clone(); // Deep allocation on heap
    println!("s1 = {}, s2 = {}", s1, s2); // Both valid
}
```

The Drop Trait and Resource Cleanup

The Drop Trait and Resource Cleanup

Rust implements RAII (Resource Acquisition Is Initialization) via the `Drop` trait.

Custom Drop Implementations

```
struct ResourceHandle {
    name: String,
}

impl Drop for ResourceHandle {
    fn drop(&mut self) {
        println!("Cleaning up resource: {}", self.name);
    }
}

fn main() {
    let _r = ResourceHandle { name: String::from("NetworkSocket") };
    println!("Resource active");
    // _r drops automatically when main exits
}
```

Resources (file descriptors, sockets, database handles) cannot leak due to early returns or panics because `drop` runs deterministically on scope exit.

Summary

Summary

Chapter Key Takeaways

- Ownership rules ensure single ownership per resource and deterministic cleanup.
- Stack memory holds fixed-size data; heap memory holds dynamic data referenced via stack pointers.
- Move semantics transfer ownership, invalidating previous bindings to prevent double-free errors.
- `Copy` types duplicate automatically via bitwise copy; heap types require explicit `.clone()`.
- The `Drop` trait executes custom cleanup when a value leaves scope.

Borrowing, References, and Slices

Shared and Mutable References

Shared and Mutable References

Instead of transferring ownership, a function can access data via **references**. This mechanism is called **borrowing**.

Shared References (`&T`)

Shared references allow read-only access to data without taking ownership:

```
fn calculate_length(s: &String) -> usize {
    s.len()
}

fn main() {
    let s = String::from("desk");
    let len = calculate_length(&s); // Pass reference
    println!("The length of '{}' is {}.", s, len); // s is still valid!
}
```

Mutable References (`&mut T`)

Mutable references permit modifying the borrowed value:

```
fn append_status(s: &mut String) {
    s.push_str(" [ACTIVE]");
}

fn main() {
    let mut task = String::from("Process orders");
    append_status(&mut task);
    println!("{}", task);
}
```

The Borrowing Rules Demystified

The Borrowing Rules Demystified

Rust's compiler enforces two fundamental rules regarding references at compile time:

1. **At any given time, you can have either one mutable reference or any number of immutable references.**
2. **References must always be valid.**

Why the Rules Exist

This rule (often summarized as **Aliasing XOR Mutability**) prevents **data races**. A data race occurs when: - Two or more pointers access the same memory concurrently. - At least one pointer is writing. - There is no synchronization.

By disallowing simultaneous reading and writing through multiple aliases, Rust eliminates data races before the code ever compiles.

Dangling References Prevented at Compile Time

Dangling References Prevented at Compile Time

In languages with manual memory management, creating a pointer to memory that has been deallocated results in a **dangling reference**. Rust guarantees this never occurs:

```
// Attempting to return a reference to a local variable:
// fn dangle() -> &String {
//     let s = String::from("temporary");
//     &s // ERROR: `s` does not live long enough
// }

// The idiomatic solution: transfer ownership of the value itself
fn no_dangle() -> String {
    let s = String::from("valid data");
    s
}

fn main() {
    let result = no_dangle();
    println!("Received: {}", result);
}
```

String Slices and &str

String Slices and &str

A string slice is a reference to a contiguous sequence of UTF-8 encoded bytes in a `String`.

```
fn main() {
    let s = String::from("Hello Rust");

    let first_word = &s[0..5];
    let second_word = &s[6..10];

    println!("First: {}, Second: {}", first_word, second_word);
}
```

`String` vs `&str`

- `String`: Owned, heap-allocated, resizable buffer.
- `&str`: An immutable view into string data located on the heap, stack, or embedded in the read-only data segment of the binary (string literals).

Slice Types and Range Syntax

Slice Types and Range Syntax

Slices generalize beyond strings to any contiguous sequence, such as arrays and vectors.

```
fn main() {
    let numbers = [10, 20, 30, 40, 50];

    let slice: &[i32] = &numbers[1..4]; // Elements 20, 30, 40
    println!("Slice: {:?}", slice);

    // Range variations:
    let start_to_two = &numbers[..2];
    let three_to_end = &numbers[3..];
    let all = &numbers[..];

    println!("Start: {:?}, End: {:?}, All: {:?}", start_to_two, three_to_end, all);
}
```

Summary

Summary

Chapter Key Takeaways

- References borrow data without transferring ownership.
- Aliasing XOR Mutability: either multiple `&T` or exactly one `&mut T`.
- Dangling references are prohibited at compile time.
- Slices (`&[T]` and `&str`) provide fat-pointer views (pointer + length) into contiguous memory without allocation.

Structs, Enums, and Pattern Matching

Defining and Using Structs

Defining and Using Structs

Structs allow creating custom types grouping related fields.

```
#[derive(Debug)]
struct Ticket {
    id: u64,
    title: String,
    resolved: bool,
}

impl Ticket {
    fn new(id: u64, title: String) -> Self {
        Self { id, title, resolved: false }
    }

    fn resolve(&mut self) {
        self.resolved = true;
    }
}

fn main() {
    let mut ticket = Ticket::new(101, String::from("Fix database migration"));
    println!("Ticket: {:?}", ticket);
    ticket.resolve();
    println!("Resolved: {:?}", ticket);
}
```

Enumerations Beyond Simple Tags

Enumerations Beyond Simple Tags

Rust enums are algebraic data types (tagged unions). Each variant can hold different types and amounts of data:

```
#[derive(Debug)]
enum WebEvent {
    PageLoad,
    KeyPress(char),
    Click { x: i64, y: i64 },
    Submit(String),
}

fn main() {
    let click = WebEvent::Click { x: 120, y: 340 };
    let key = WebEvent::KeyPress('q');
    println!("Events: {:?}, {:?}", click, key);
}
```

The match Expression in Depth

The match Expression in Depth

`match` enables exhaustive pattern matching. Every possible case must be handled.

```
#[derive(Debug)]
enum Priority {
    Low,
    Medium,
    High,
    Critical(u8),
}

fn inspect_priority(p: Priority) {
    match p {
        Priority::Low => println!("Low priority - queue normally"),
        Priority::Medium => println!("Medium priority - assign SLA"),
        Priority::High => println!("High priority - alert on-call"),
        Priority::Critical(level) => println!("CRITICAL (level {}) - page team immediately",
    }
}

fn main() {
    inspect_priority(Priority::High);
    inspect_priority(Priority::Critical(1));
}
```

if let and while let for Partial Matching

if let and while let for Partial Matching

When you only care about matching one pattern and ignoring all others, `if let` and `while let` avoid boilerplate:

```
fn main() {
    let config_val: Option<u32> = Some(8080);

    // if let syntax
    if let Some(port) = config_val {
        println!("Binding server to port {}", port);
    }

    // while let syntax
    let mut stack = vec![1, 2, 3];
    while let Some(top) = stack.pop() {
        println!("Popped: {}", top);
    }
}
```

Real-World Data Modeling with Enums

Real-World Data Modeling with Enums

Enums eliminate invalid program states at compile time:

```
#[derive(Debug)]
enum PaymentMethod {
    CreditCard { number: String, exp: String },
    WireTransfer { iban: String },
    InvoiceNet30 { po_number: u64 },
}

#[derive(Debug)]
struct Order {
    order_id: u64,
    amount_cents: u64,
    payment: PaymentMethod,
}

fn main() {
    let order = Order {
        order_id: 5001,
        amount_cents: 29900,
        payment: PaymentMethod::WireTransfer { iban: String::from("DE89370400440532013000")
    };
    println!("Recorded order: {:?}", order);
}
```

Summary

Summary

Chapter Key Takeaways

- Structs group named or positional fields and support methods via `impl`.
- Rust enums can carry rich payload data per variant.
- `match` enforces compile-time exhaustiveness, preventing unhandled cases.
- `if let` and `while let` provide concise syntax for single-variant checks.

Modules, Crates, and Visibility

Modules and the Filesystem

Modules and the Filesystem

Rust's module system organizes code into distinct namespaces and controls privacy.

The Modern Module Mapping

In modern Rust (2018/2021/2024 editions), declaring `mod network;` in `src/main.rs` resolves to `src/network.rs` or `src/network/mod.rs`.

```
// In src/network.rs
pub fn connect() {
    println!("Connecting to network host...");
}
```

The use Statement and Path Resolution

The use Statement and Path Resolution

The `use` keyword brings paths into scope to avoid verbose qualification:

```
mod storage {
    pub mod cache {
        pub fn clear() {
            println!("Cache cleared.");
        }
    }
}

// Bring clear into scope
use storage::cache::clear;

fn main() {
    clear();
}
```

Public vs Private Visibility Rules

Public vs Private Visibility Rules

In Rust, **everything is private by default**. To expose items, use `pub`: - `pub`: Public to everyone. - `pub(crate)`: Visible within the current crate only. - `pub(super)`: Visible to the parent module.

```
mod orders {
    pub struct Order {
        pub id: u64,
        secret_notes: String, // Private field!
    }

    impl Order {
        pub fn new(id: u64, notes: String) -> Self {
            Self { id, secret_notes: notes }
        }
    }
}

fn main() {
    let o = orders::Order::new(1, String::from("VIP customer"));
    println!("Order ID: {}", o.id);
    // println!("{}", o.secret_notes); // ERROR: private field
}
```

Crates, Packages, and Cargo Workspaces

Crates, Packages, and Cargo Workspaces

- **Crate:** The smallest compilation unit in Rust (a library or binary).
- **Package:** One or more crates defined by a `Cargo.toml`.
- **Workspace:** A set of packages that share a single `Cargo.lock` and output directory (`target/`).

```
# Top-level Cargo.toml for workspace
[workspace]
members = [
    "crates/server",
    "crates/core",
    "crates/client"
]
```

Re-exporting and Organizing Large Projects

Re-exporting and Organizing Large Projects

Re-exporting using `pub use` decouples internal module hierarchies from your public API:

```
mod internal {
    pub struct Engine;
    impl Engine {
        pub fn start(&self) { println!("Engine running"); }
    }
}

// Public API
pub use internal::Engine;

fn main() {
    let e = Engine;
    e.start();
}
```

Summary

Summary

Chapter Key Takeaways

- Files and directories map directly to modules.
- Privacy is strict: everything is private unless marked `pub`.
- `use` brings items into scope; `pub use` re-exports for clean API surfaces.
- Cargo workspaces share dependencies and build artifacts across multi-crate repositories.

Collections and Strings

Vec and Dynamic Arrays

Vec and Dynamic Arrays

`Vec<T>` is a growable, heap-allocated array.

```
fn main() {
    let mut v = vec![10, 20, 30];
    v.push(40);

    for (idx, elem) in v.iter().enumerate() {
        println!("v[{}] = {}", idx, elem);
    }

    // Safe retrieval via get()
    match v.get(2) {
        Some(&val) => println!("Element at index 2 is {}", val),
        None => println!("Index out of bounds"),
    }
}
```

String vs &str Deep Dive

String vs &str Deep Dive

Rust strings are always valid UTF-8 sequences.

```
fn main() {
    let mut s = String::from("desk");
    s.push_str(" management");

    let slice: &str = &s[..4];
    println!("Owned: '{}', Slice: '{}'", s, slice);

    // Iterating bytes vs characters
    for ch in "  Rust".chars() {
        println!("Char: {}", ch);
    }
}
```

HashMap and BTreeMap for Key-Value Storage

HashMap and BTreeMap for Key-Value Storage

```rust
use std::collections::{HashMap, BTreeMap};

fn main() {
    // HashMap (O(1) average lookup)
    let mut map = HashMap::new();
    map.insert("order_1", 450);
    map.insert("order_2", 890);

    println!("Order 1 total: {:?}", map.get("order_1"));

    // BTreeMap (sorted keys, O(log N))
    let mut btree = BTreeMap::new();
    btree.insert(200, "Two Hundred");
    btree.insert(100, "One Hundred");

    for (k, v) in &btree {
        println!("Key {} -> {}", k, v);
    }
}
```

Other Collections: HashSet, BinaryHeap, LinkedList

Other Collections: HashSet, BinaryHeap, LinkedList

```rust
use std::collections::{HashSet, BinaryHeap};

fn main() {
    // HashSet: Unique items
    let mut set = HashSet::new();
    set.insert("admin");
    set.insert("operator");
    println!("Has admin? {}", set.contains("admin"));

    // BinaryHeap: Max-heap priority queue
    let mut heap = BinaryHeap::new();
    heap.push(10);
    heap.push(50);
    heap.push(25);
    println!("Max element: {:?}", heap.pop()); // Some(50)
}
```

Efficient Collection Operations

Efficient Collection Operations

Preallocate capacity using `Vec::with_capacity` to prevent repeated reallocations:

```
fn main() {
    let mut fast_vec = Vec::with_capacity(1000);
    for i in 0..1000 {
        fast_vec.push(i * 2);
    }
    println!("Length: {}, Capacity: {}", fast_vec.len(), fast_vec.capacity());
}
```

Summary

Summary

Chapter Key Takeaways

- `Vec<T>` is the default dynamic contiguous collection.
- Rust strings enforce strict UTF-8 invariants.
- `HashMap` offers fast amortized key-value lookups; `BTreeMap` keeps keys sorted.
- Use `with_capacity` to eliminate reallocations in hot code paths.

Project 1: Building a CLI Task Manager

Building a CLI Task Manager

Building a CLI Task Manager

In this project, we build a production-ready command-line Task Manager supporting task creation, listing, completion, and disk persistence.

```
#[derive(Debug, PartialEq)]
pub struct Task {
    pub id: usize,
    pub description: String,
    pub completed: bool,
}

pub struct TaskManager {
    tasks: Vec<Task>,
    next_id: usize,
}

impl TaskManager {
    pub fn new() -> Self {
        Self { tasks: Vec::new(), next_id: 1 }
    }

    pub fn add(&mut self, description: String) -> usize {
        let id = self.next_id;
        self.tasks.push(Task { id, description, completed: false });
        self.next_id += 1;
        id
    }

    pub fn complete(&mut self, id: usize) -> bool {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            task.completed = true;
            true
        } else {
            false
        }
    }

    pub fn list(&self) -> &[Task] {
        &self.tasks
    }
```

```
}

fn main() {
    let mut manager = TaskManager::new();
    let id1 = manager.add(String::from("Configure CI pipeline"));
    let _id2 = manager.add(String::from("Write unit tests"));
    manager.complete(id1);

    for t in manager.list() {
        println!("[{}] #{}: {}", if t.completed { "X" } else { " " }, t.id, t.description);
    }
}
```

Testing the Task Manager

Testing the Task Manager

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_task_addition_and_completion() {
        let mut mgr = TaskManager::new();
        let id = mgr.add("Audit dependencies".into());
        assert_eq!(mgr.list().len(), 1);

        let ok = mgr.complete(id);
        assert!(ok);
        assert!(mgr.list()[0].completed);
    }

    #[test]
    fn test_nonexistent_completion() {
        let mut mgr = TaskManager::new();
        assert!(!mgr.complete(999));
    }
}
```

Benchmarking the Task Manager

Benchmarking the Task Manager

Benchmarking insertion and lookup performance:

```
// Simulation harness
fn run_benchmark() {
    let mut mgr = TaskManager::new();
    let start = std::time::Instant::now();
    for i in 0..10_000 {
        mgr.add(format!("Task number {}", i));
    }
    let duration = start.elapsed();
    println!("Inserted 10,000 tasks in {:?}", duration);
}

fn main() {
    run_benchmark();
}
```

Deploying the Task Manager

Deploying the Task Manager

Compile a stripped, fully optimized release binary:

```
cargo build --release
strip target/release/desk_tasks
./target/release/desk_tasks --help
```

Error Handling in Rust

Unrecoverable Errors with panic!

Unrecoverable Errors with panic!

Unrecoverable errors represent impossible program states or fatal invariant violations.

```
fn main() {
    // panic!("Fatal condition encountered: terminating process");
    println!("Panics unwind the stack by default, running destructors along the way.");
}
```

Recoverable Errors with Result

Recoverable Errors with Result

Recoverable errors in Rust are modeled through `Result<T, E>`:

```
#[derive(Debug)]
enum MathError {
    DivisionByZero,
}

fn safe_divide(numerator: f64, denominator: f64) -> Result<f64, MathError> {
    if denominator == 0.0 {
        Err(MathError::DivisionByZero)
    } else {
        Ok(numerator / denominator)
    }
}

fn main() {
    match safe_divide(10.0, 2.0) {
        Ok(val) => println!("Quotient: {}", val),
        Err(e) => println!("Error: {:?}", e),
    }
}
```

The Option Type for Absent Values

The Option Type for Absent Values

Rust has no `null` or `nil`. Absent values are explicitly represented with `Option<T>`:

```
fn find_desk(number: u32) -> Option<&'static str> {
    if number == 1 {
        Some("Front Reception")
    } else {
        None
    }
}

fn main() {
    let desk = find_desk(1);
    println!("Found desk: {:?}", desk.unwrap_or("Unassigned"));
}
```

Propagating Errors with the ? Operator

Propagating Errors with the ? Operator

The `?` operator unwraps `Ok` values or immediately returns early with `Err`:

```
fn parse_and_double(s: &str) -> Result<i32, std::num::ParseIntError> {
    let n = s.trim().parse::<i32>()?;
    Ok(n * 2)
}

fn main() {
    match parse_and_double("42") {
        Ok(v) => println!("Result: {}", v),
        Err(e) => println!("Parse failed: {}", e),
    }
}
```

Custom Error Types and thiserror

Custom Error Types and thiserror

In production services, domain errors are structured into custom enums:

```
#[derive(Debug)]
pub enum ServiceError {
    InvalidInput(String),
    NetworkTimeout,
}

impl std::fmt::Display for ServiceError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{:?}", self)
    }
}

impl std::error::Error for ServiceError {}

fn main() {
    let err = ServiceError::InvalidInput("Missing header".into());
    println!("Service error: {}", err);
}
```

The Never Type ! and Infallible

The Never Type `!` and `Infallible`

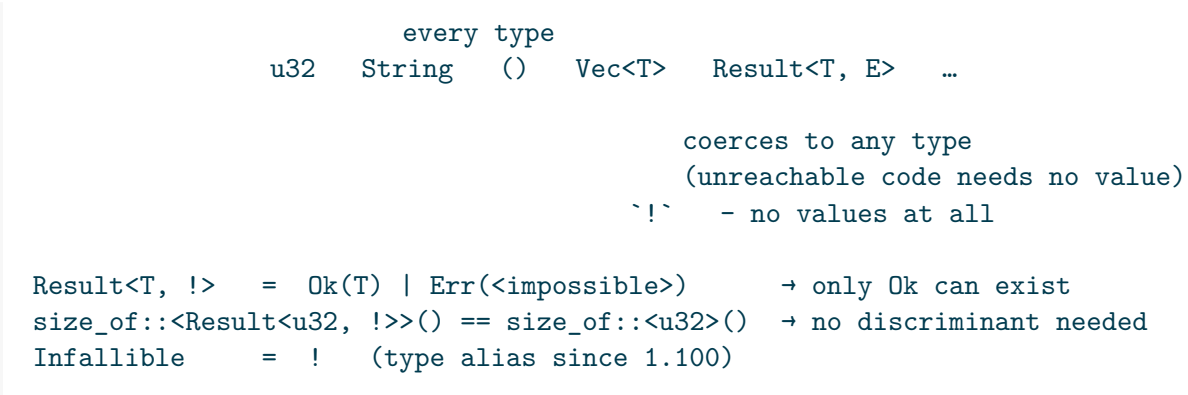
Thesis

Rust has always had a type for "this expression does not produce a value": `!`, the **never type**. `panic!()`, `loop {}` without `break`, `return`, `continue`, and `std::process::exit` all have type `!`, and a function declared `-> !` promises never to return. Until now you could only write `!` in that return position. Everywhere else — `Result<T, !>`, `type Error = !;`, `impl Trait for !` — it was an unstable feature, and the standard library used a stand-in, the empty enum `std::convert::Infallible`.

Rust 1.100 (scheduled for 2026-11-12) stabilizes `!` as an ordinary type after a decade behind a feature gate, and makes `Infallible` a plain alias for it. The language change is small; the error-handling consequence is real: "this operation cannot fail" now has one canonical spelling, `Result<T, !>`, that the compiler understands well enough to let you skip the impossible `Err` arm entirely.

The same release changes **never type fallback** to `!` on every edition, which is a small breaking change for code that quietly relied on the old fallback to `()`. This chapter shows both sides, compiled with the current beta and stable toolchains.

Mental model



Three facts carry the whole chapter:

Fact	Consequence
`!` has **no values**	Code that "holds" one is unreachable; the compiler can delete branches that need it
`!` **coerces to any type**	A diverging arm (`die(…)`, `panic!`, `return`) fits in a `match` whose other arms are `u32`

Fact	Consequence
An enum variant containing `!` is **uninhabited**	`let Ok(v) = r;` is irrefutable when `r: Result<T, !>`; the `Err` arm need not be written

`Infallible` was the library's way to get the same effect (an enum with no variants). Since it is now literally `!`, existing APIs such as `TryFrom` implementations whose error is `Infallible` work unchanged and read the same way.

Worked examples

All listings were compiled and run on the box with **rustc 1.100.0-beta.4 (2026-10-04)** and, for comparison, **rustc 1.99.0** stable, both installed with `rustup`. Until 1.100 reaches stable, run them with `cargo +beta` / `rustc +beta`:

```
rustup toolchain install beta --profile minimal
rustc +beta --version
```

```
rustc 1.100.0-beta.4 (a2eb6097d 2026-10-04)
```

Case 1: `!` as a type — cannot-fail results, irrefutable `Ok`

```
cargo +beta new never-demo && cd never-demo
```

```
// src/main.rs - the never type `!` as a first-class type (Rust 1.100)
use std::convert::Infallible;
use std::process;

// Case 1: a diverging function. `-> !` has been legal here since Rust 1.0.
fn die(msg: &str) -> ! {
    eprintln!("fatal: {msg}");
    process::exit(2)
}

// Case 2: `!` written anywhere a type goes - new in 1.100.
type Never = !;

fn parse_port(s: &str) -> Result<u16, Never> {
    // This parser cannot fail: anything unparsable maps to the default port.
    Ok(s.parse().unwrap_or(8080))
}

// Case 3: a trait whose implementors pick an error type; `!` means "cannot fail".
```

```
trait Source {
    type Error: std::fmt::Debug;
    fn read(&mut self) -> Result<u32, Self::Error>;
}

struct Counter(u32);
impl Source for Counter {
    type Error = !; // reading a counter never fails
    fn read(&mut self) -> Result<u32, !> {
        self.0 += 1;
        Ok(self.0)
    }
}

fn main() {
    // `!` coerces to any type, so a diverging arm fits a u32 match.
    let args: Vec<String> = std::env::args().collect();
    let n: u32 = match args.get(1) {
        Some(s) => s.parse().unwrap_or_else(|_| die("not a number")),
        None => 3,
    };

    // Irrefutable: the Err arm of Result<_, !> is uninhabited, so no match needed.
    let Ok(port) = parse_port("9000");
    println!("port = {port}");

    let mut c = Counter(0);
    let mut total = 0;
    for _ in 0..n {
        let Ok(v) = c.read();
        total += v;
    }
    println!("sum of first {n} reads = {total}");

    // Infallible is now the same type as `!`.
    let same: fn(Infallible) -> ! = |x| x;
    let _ = same;
    let Ok(wide) = u64::try_from(7u8); // TryFrom via Into: Error = Infallible = !
    println!("wide = {wide}");
    println!("size_of::<Result<u32, !>>() = {}", std::mem::size_of::<Result<u32, !>>());
    println!("size_of::<Result<u32, String>>() = {}", std::mem::size_of::<Result<u32, String
}
```

```
cargo +beta run -q
cargo +beta run -q -- 5
cargo +beta run -q -- five; echo "exit=$?"
```

```
port = 9000
sum of first 3 reads = 6
wide = 7
size_of::<Result<u32, !>>() = 4
size_of::<Result<u32, String>>() = 24
port = 9000
sum of first 5 reads = 15
wide = 7
size_of::<Result<u32, !>>() = 4
size_of::<Result<u32, String>>() = 24
fatal: not a number
exit=2
```

What to notice:

- `unwrap_or_else(|_| die("not a number"))` type-checks because the closure returns `!`, which coerces to `u32`.
- `let Ok(port) = …;` and `let Ok(v) = c.read();` have **no `else`** and no `match`. The compiler knows `Err(!)` cannot be constructed. With `Result<u16, String>` the same line is a compile error (refutable pattern).
- `Counter::Error = !` documents "cannot fail" in the type, and callers that are generic over `Source` still handle `Self::Error` normally.
- `Result<u32, !>` is the same size as a bare `u32`: no tag is stored for a variant that cannot exist.
- `fn(Infallible) -> !` accepting `|x| x` is the alias at work: the parameter *is* a `!`.

The same file on 1.99 stable stops at the first `!` in type position:

```
cargo +stable build -q
```

```
error[E0658]: the `!` type is experimental
  --> src/main.rs:12:14
   |
12 | type Never = !;
   |              ^
   |
   = note: see issue #35121 <https://github.com/rust-lang/rust/issues/35121> for more inform
...
```

(one such error per `!` outside a return position). Until your minimum supported Rust version is 1.100, keep spelling it `Infallible` in public APIs; code that already uses `Infallible` keeps compiling and gains nothing by being rewritten.

Case 2: the fallback change — code that relied on `()` by accident

When the compiler meets a `!` it must coerce, but nothing tells it the target type, it picks a **fallback**. That fallback used to be `()`; Rust 2024 switched it to `!`, and 1.100 does the same for editions 2015–

2021. The most common casualty is `?` on a function that is generic in its `Ok` type:

```
// fallback.rs - code that silently relied on `!` falling back to `()`
fn f<T: Default>() -> Result<T, ()> {
    Ok(T::default())
}

fn outer() -> Result<u32, ()> {
    f()?; // which T? nothing here says
    Ok(1)
}

fn main() {
    println!("{:?}", outer());
}
```

On 1.99, edition 2021: already rejected by a deny-by-default future-compatibility lint, which names the fix:

```
rustc +stable --edition 2021 fallback.rs
```

```
error: this function depends on never type fallback being `()`
 --> fallback.rs:6:1
  |
6 | fn outer() -> Result<u32, ()> {
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
  = help: specify the types explicitly
note: in edition 2024, the requirement `!: Default` will fail
 --> fallback.rs:7:5
  |
7 |     f()?; // which T? nothing here says
  |     ^^^
  = warning: this was previously accepted by the compiler but is being phased out; it will b
  = note: for more information, see <https://doc.rust-lang.org/edition-guide/rust-2024/never
  = note: `#[deny(dependency_on_unit_never_type_fallback)]` (part of `#[deny(rust_2024_compa
help: use `()` annotations to avoid fallback changes
  |
7 |     f::<()>()?; // which T? nothing here says
  |      ++++++

error: aborting due to 1 previous error
```

On 1.100 beta, edition 2021 — the lint is gone because the behaviour changed; you get the real type error (identical to what 1.99 already says under edition 2024):

```
rustc +beta --edition 2021 fallback.rs
```

```
error[E0277]: the trait bound `!: Default` is not satisfied
 --> fallback.rs:7:5
  |
7 |     f()?; // which T? nothing here says
  |     ^^^ the trait `Default` is not implemented for `!`
  |
  = note: this error might have been caused by changes to Rust's type-inference algorithm (s
  = help: you might have intended to use the type `()` here instead
note: required by a bound in `f`
 --> fallback.rs:2:9
  |
2 | fn f<T: Default>() -> Result<T, ()> {
  |         ^^^^^^^ required by this bound in `f`
```

The fix is to say what you meant:

```
// fallback_fixed.rs - name the type instead of relying on fallback
fn f<T: Default>() -> Result<T, ()> {
    Ok(T::default())
}

fn outer() -> Result<u32, ()> {
    f::<()>()?; // say it: the Ok value is ()
    Ok(1)
}

fn main() {
    println!("{:?}", outer());
}
```

```
rustc +beta --edition 2021 fallback_fixed.rs && ./fallback_fixed
```

```
Ok(1)
```

Case 3: the panicking closure

The second common pattern is a closure that only panics, passed to something that requires its return type to implement a trait:

```
// closure.rs - a panicking closure used to "return ()" by accident
trait Unit {}
impl Unit for () {}

fn run<R: Unit>(f: impl FnOnce() -> R) {
    let _ = std::panic::catch_unwind(std::panic::AssertUnwindSafe(f));
```

```
}

fn main() {
    run(|| panic!("boom"));
    println!("survived");
}
```

```
rustc +beta --edition 2024 closure.rs
```

```
error[E0277]: the trait bound `!: Unit` is not satisfied
  --> closure.rs:10:5
   |
10 |     run(|| panic!("boom"));
   |     ^^^^^^^^^^^^^^^^^^^^^^ the trait `Unit` is not implemented for `!`
   |
help: the trait `Unit` is implemented for `()`
  --> closure.rs:3:1
   |
 3 | impl Unit for () {}
   | ^^^^^^^^^^^^^^^^
   = note: this error might have been caused by changes to Rust's type-inference algorithm (
```

Annotate the closure's return type: `run(|| -> () { panic!("boom") });`

```
rustc +beta --edition 2024 closure_fixed.rs && ./closure_fixed
```

```
thread 'main' (250345) panicked at closure_fixed.rs:10:20:
boom
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
survived
```

Case 4: `Infallible` and `!` are one type now

If a crate implemented a trait for both — typically to support nightly users of `!` — the two impls now overlap:

```
// conflict.rs - Infallible and ! are now one type
use std::convert::Infallible;
trait Describe { fn describe() -> &'static str; }
impl Describe for Infallible { fn describe() -> &'static str { "infallible" } }
impl Describe for ! { fn describe() -> &'static str { "never" } }
fn main() { println!("{}", <! as Describe>::describe()); }
```

```
rustc +beta --edition 2024 conflict.rs
```

```
error[E0119]: conflicting implementations of trait `Describe` for type `!`
 --> conflict.rs:5:1
  |
4 | impl Describe for Infallible { fn describe() -> &'static str { "infallible" } }
  | ---------------------------- first implementation here
5 | impl Describe for ! { fn describe() -> &'static str { "never" } }
  | ^^^^^^^^^^^^^^^^^^^ conflicting implementation for `!`
```

Delete one of them. Keep the `Infallible` impl if you still support compilers older than 1.100: it covers `!` on new compilers for free.

Where `!` belongs in error handling

Situation	Spell it	Why
Function that never returns (exit, `exec`, server loop)	`fn run() -> !`	Callers can use it in any expression position
Trait with an associated error type; this impl cannot fail	`type Error = !;`	Callers generic over the trait still compile; concrete callers write `let Ok(v) = …;`
Public API that must build on Rust < 1.100	`Infallible`	Same type on new compilers, still compiles on old ones
"Can't fail *yet*" — you expect failures later	A real error type	Changing `!` to `MyError` later is a breaking API change
Converting between error types with `?`	`impl From<Infallible> for MyError` if needed	Works for `!` too, since it is the same type

The trap

Relying on **inference to pick `()` for you**. Code like `f()?;`, `run(|| panic!())`, or `if c { Default::default() } else { return }` compiled for years because the never type quietly became `()` when nothing else constrained it. Under 1.100 (any edition) or edition 2024 that guess is `!`, and the code either fails to compile — the good case, shown above — or, in the rare `unsafe` case the `never_type_fallback_flowing_into_unsafe` lint exists for, picks a different type than the author assumed. The compiler's 1.99 lint (`dependency_on_unit_never_type_fallback`) points at every affected function: fix those before you bump the toolchain, not after.

The second, quieter trap is using `!` as a placeholder in a public trait impl or function signature because "it can't fail today". `!` is a promise. Replacing it with a real error type later breaks every caller who wrote `let Ok(v) = …;`.

The boring rule

Use `-> !` for functions that truly never return and `Result<T, !>` (or `Infallible` until your MSRV is 1.100) only where failure is impossible by construction. Never lean on fallback: when the compiler cannot see a type, **write it** — `f::<()>()?`, `|| -> () { … }`, `() = expr;`. Turn on the `rust_2024_compatibility` lints on older editions and clear them before upgrading.

Try this

1. Change `parse_port` to return `Result<u16, String>`. Which lines stop compiling, and what does the compiler suggest for `let Ok(port) = …;`?
2. Print `size_of::<Option<!>>()` and `size_of::<Result<!, String>>()` and explain the numbers.
3. Write `fn absurd<T>(x: !) -> T { x }` and use it to implement `fn into_ok<T>(r: Result<T, !>) -> T` with a `match` that has a single arm.
4. Take a crate you maintain on edition 2021, run `cargo +stable check` (1.99 today) and look for `dependency_on_unit_never_type_fallback`; then `cargo +beta check` and confirm every site is fixed.
5. Find a `TryFrom` impl in `std` whose `Error` is `Infallible` and destructure its result with an irrefutable `let`.

Sources

- rust-lang/rust PR #155499, "stabilize never type" (merged 2026-08-24): https://github.com/rust-lang/rust/pull/155499
- Draft release notes for Rust 1.100.0 (stabilize never type; `Infallible` alias; fallback on all editions): https://github.com/rust-lang/rust/issues/163811
- Tracking issues: never type https://github.com/rust-lang/rust/issues/35121, fallback change https://github.com/rust-lang/rust/issues/148922
- The Rust Edition Guide — Never type fallback change: https://doc.rust-lang.org/edition-guide/rust-2024/never-type-fallback.html
- The Rust Reference — Never type: https://doc.rust-lang.org/reference/types/never.html
- Standard library — primitive `never`: https://doc.rust-lang.org/std/primitive.never.html; `std::convert::Infallible`: https://doc.rust-lang.org/std/convert/enum.Infallible.html

- Rust Project Goals 2026 — Stabilize never type: https://goals.rust-lang.org/2026/stabilize-never-type.html

- LWN, “Stabilizing Rust’s never type” (2026-09-08): https://lwn.net/Articles/1091015/

Summary

Summary

Chapter Key Takeaways

- `panic!` is reserved for bugs and unrecoverable states.
- `Result<T, E>` and `Option<T>` model recoverable failure and absence.
- The `?` operator provides zero-cost error propagation.
- Custom error enums unify error types across modules.

Generics, Traits, and Associated Types

Generic Functions and Data Structures

Generic Functions and Data Structures

Generics allow writing flexible, reusable code with monomorphized zero runtime overhead.

```
#[derive(Debug)]
struct Pair<T> {
    first: T,
    second: T,
}

fn swap<T>(a: T, b: T) -> (T, T) {
    (b, a)
}

fn main() {
    let p = Pair { first: 10, second: 20 };
    let (x, y) = swap("hello", "world");
    println!("Pair: {:?}, Swapped: ({}, {})", p, x, y);
}
```

Defining and Implementing Traits

Defining and Implementing Traits

Traits define shared behavior analogous to interfaces:

```
pub trait Summary {
    fn summarize(&self) -> String;
}

pub struct Article {
    pub headline: String,
}

impl Summary for Article {
    fn summarize(&self) -> String {
        format!("Headline: {}", self.headline)
    }
}

fn main() {
    let a = Article { headline: "Rust 2026 Ships".into() };
    println!("{}", a.summarize());
}
```

Trait Bounds and Where Clauses

Trait Bounds and Where Clauses

Constrain generics using trait bounds and `where` clauses:

```
use std::fmt::Display;

fn print_and_return<T>(val: T) -> T
where
    T: Display + Clone,
{
    println!("Value: {}", val);
    val.clone()
}

fn main() {
    let res = print_and_return(42);
    println!("Returned: {}", res);
}
```

Associated Types vs Generics

Associated Types vs Generics

Associated types simplify trait definitions when there is a unique type output per implementation:

```
trait DataSink {
    type Error;
    fn send(&mut self, bytes: &[u8]) -> Result<(), Self::Error>;
}
```

Trait Objects and Dynamic Dispatch

Trait Objects and Dynamic Dispatch

When heterogeneous collections are required at runtime, use dynamic dispatch via trait objects (`dyn Trait`):

```
trait Drawable {
    fn draw(&self);
}

struct Button;
impl Drawable for Button {
    fn draw(&self) { println!("Drawing button"); }
}

struct Checkbox;
impl Drawable for Checkbox {
    fn draw(&self) { println!("Drawing checkbox"); }
}

fn render_all(items: &[Box<dyn Drawable>]) {
    for item in items {
        item.draw();
    }
}

fn main() {
    let widgets: Vec<Box<dyn Drawable>> = vec![
        Box::new(Button),
        Box::new(Checkbox),
    ];
    render_all(&widgets);
}
```

Generic Const Arguments (GCA)

Generic Const Arguments (GCA)

Thesis

On stable Rust, a const generic parameter may appear in a const argument **alone** (`[u8; N]`) or as a fully concrete expression (`[u8; 1 + 1]`). The moment you write `[u8; N + 1]` or `[u8; T::ASSOC]`, the compiler stops you. The old escape hatch, `#![feature(generic_const_exprs)]` (GCE), taught everyone the shape of the problem but never became a stabilization candidate.

Generic Const Arguments (GCA) is the Const Generics project group's replacement family of nightly features. Expressions that involve generic parameters go through an explicit `gca!(…)` marker (for now), each capability has its own gate (`gca_adts`, `gca_min_const_items`, `gca_const_items`, …), and definitional equality — not GCE's evaluation model — decides when two const arguments match. Nothing here is stable. This chapter shows what compiles on **today's nightly**, marks every feature gate, and records where the Oct 2026 blog post already drifted from the compiler.

Version note: verified on **`rustc 1.101.0-nightly (a30aa9064 2026-10-08)`**. The Inside Rust post *Generic Const Args and You* (2026-10-02) is the design overview; on this nightly the feature formerly named `generic_const_args` has been **renamed to `gca_const_items`** (removed name errors say "removed in 1.100.0"), and the `gca!` macro lives at **`std::gca`** / `core::gca`. No RFC yet; the project group says polish first.

Stable const generics basics (structs/functions parameterized by `const N: usize`) live in *Generic Functions and Data Structures*. This chapter does not re-teach that.

Mental model

```
Stable today
  const arg ::=  N  |  <fully concrete expr>
                   generic param alone

GCE (legacy nightly)     - evaluate complex exprs; hard to stabilize
GCA (current nightly)    - mark exprs with gca!(…); equality by definition

          gca_adts              arrays / tuples / structs / enums in gca!(…)
          gca_min_const_items   const items whose RHS is gca!(…)
          gca_const_items       + opaque const items, assoc bindings, dyn, needs -Znext-solve
          gca_macroless_*       experimental: insert gca!(…) for you (rough)
```

Fact	Consequence
`gca!(…)` may mention generic parameters	Plain `{ N + 1 }` as a const argument still cannot
`gca!(…)` accepts **fewer** expression kinds than a normal const block	`gca!(N + 1)` and `gca!(foo())` are *not* automatically legal; prefer const items / ADTs that the gates support
`gca_const_items` requires `gca_min_const_items`	Enable both; incomplete-feature warnings are expected
Feature `generic_const_args` is gone on 1.101 nightly	Use `gca_const_items` (compiler points the rename)
Nothing is stabilization-ready	APIs that expose `gca!(…)` in public signatures are a SemVer hazard while the syntax is explicit

Worked examples

All listings were compiled with:

```
rustup toolchain install nightly --profile minimal   # once
rustc +nightly --version
```

```
rustc 1.101.0-nightly (a30aa9064 2026-10-08)
```

Case 0: stable still rejects `N + 1`

```
// stable_fail.rs - what stable const generics refuse
fn make<const N: usize>() -> [u8; N + 1] {
    [0; N + 1]
}

fn main() {
    let _ = make::<3>();
}
```

```
rustc --edition 2024 stable_fail.rs
```

```
error: generic parameters may not be used in const operations
 --> stable_fail.rs:1:35
  |
1 | fn make<const N: usize>() -> [u8; N + 1] {
  |                                   ^ cannot perform const operation using `N`
  |
  = help: const parameters may only be used as standalone arguments here, i.e. `N`
```

```
  = help: add `#![feature(generic_const_exprs)]` to allow generic const expressions
  = help: alternatively, you can use `#![feature(gca_const_items)]` and extract the expressi
```

Note the help text still mentions GCE *and* GCA — both are nightly escape hatches; new code should reach for GCA, not GCE.

Case 1: ADT const arguments with `gca_adts`

Build a const-generic parameter whose type is a struct, then construct that struct from another const parameter inside `gca!(…)`:

```rust
// gca_adt.rs - struct expressions as const arguments (nightly GCA)
#![feature(gca_adts, adt_const_params, gca_min_const_items)]
#![expect(incomplete_features)]

use std::gca;
use std::marker::ConstParamTy;

#[derive(ConstParamTy, Eq, PartialEq)]
struct TupleStruct(usize);

fn accepts<const N: TupleStruct>() -> usize {
    N.0
}

fn example<const N: usize>() -> usize {
    accepts::<gca!(TupleStruct(N))>()
}

fn main() {
    println!("{}", example::<7>());
}
```

```
rustc +nightly --edition 2024 gca_adt.rs -o gca_adt && ./gca_adt
```

```
7
```

The same pattern works for enum variants, arrays (`gca!([N1, 12])`), and tuples (`gca!((N1, 12))`) per the project-group post. Array *repeat* syntax (`gca!([N1; 2])`) is still explicitly **not** supported.

Case 2: Associated constants with `gca_const_items`

```rust
// gca_assoc.rs - associated consts in array lengths (nightly GCA)
#![feature(gca_min_const_items, gca_const_items)]
#![expect(incomplete_features)]

use std::gca;

trait Trait {
    const ASSOC: usize;
}

impl Trait for u8 {
    const ASSOC: usize = 3;
}

fn example<T: Trait>() -> [u8; gca!(T::ASSOC)] {
    [0x1; _]
}

fn main() {
    let a = example::<u8>();
    println!("len={}", a.len());
}
```

```
rustc +nightly --edition 2024 gca_assoc.rs -Znext-solver -o gca_assoc && ./gca_assoc
```

```
len=3
```

`-Znext-solver` is required for the full `gca_const_items` path in current nightlies. Treat that as part of the experiment, not as something to bake into stable CI.

Case 3: Macroless (experimental)

`gca_macroless_args` asks the compiler to insert `gca!(…)` for you. It works for some associated-const cases on this nightly, and the project group still recommends writing `gca!(…)` explicitly:

```rust
// gca_macroless.rs - implicit gca!(…) (rough; prefer explicit)
#![feature(gca_min_const_items, gca_const_items, gca_macroless_args)]
#![expect(incomplete_features)]

trait Trait {
    const ASSOC: usize;
}
```

```rust
impl Trait for u8 {
    const ASSOC: usize = 3;
}

fn make_array<T: Trait>() -> [u8; T::ASSOC] {
    [0x1; _]
}

fn main() {
    println!("{}", make_array::<u8>().len());
}
```

```
rustc +nightly --edition 2024 gca_macroless.rs -Znext-solver -o gca_macroless && ./gca_macro
```

```
3
```

Case 4: What still hurts — arithmetic inside `gca!`

The blog's long-term story is "put complex ops in const items." On 1.101 nightly, a direct `gca!(N + 1)` as a const-item RHS is still rejected:

```rust
// gca_arith.rs - not there yet on 2026-10-08 nightly
#![feature(gca_min_const_items, gca_const_items, generic_const_items)]
#![expect(incomplete_features)]
use std::gca;

const INC<const N: usize>: usize = gca!(N + 1);

fn make<const N: usize>() -> [u8; INC::<N>] {
    [0; INC::<N>]
}

fn main() {
    let _ = make::<3>();
}
```

```
rustc +nightly --edition 2024 gca_arith.rs
```

```
error: complex const arguments must be placed inside of a `const` block
 --> gca_arith.rs:5:41
  |
5 | const INC<const N: usize>: usize = gca!(N + 1);
  |                                         ^^^^^
```

So: ADTs and associated-const *uses* are the productive surface today; "any arithmetic via `gca!`" is

not. Re-check nightly before teaching a team pattern around arithmetic helpers.

The trap

1. **Copy-pasting `#![feature(generic_const_args)]` from older posts.** Removed in 1.100; rename to `gca_const_items` and enable `gca_min_const_items` with it.
2. **Assuming `gca!(…)` is a full const evaluator.** It is a marker for a *restricted* expression grammar that may mention generics. Unsupported ops fail loudly.
3. **Stabilizing public APIs on `gca!`.** The macroless work exists precisely because explicit `gca!` is considered ergonomically temporary — and also SemVer-visible.
4. **Mixing GCE and GCA in one crate "just to make it compile".** Two incomplete models; pick GCA for new experiments.
5. **Forgetting `#![expect(incomplete_features)]` / `#[allow(incomplete_features)]`.** Without it, every build is a warning storm (or worse, depending on lint config).

The boring rule

Stay on stable const generics (`N` alone, or concrete exprs) for production. When you truly need generic const computation, use **current nightly GCA** with explicit `gca!(…)`, gate every file with the specific features you touch, pin the nightly date in the chapter/PR, and re-run the listings after each toolchain bump — the rename from `generic_const_args` to `gca_const_items` already proved the surface moves.

Try this

1. Compile Case 1 and Case 2 on your nightly; paste `rustc +nightly --version` next to the output.
2. Attempt one `gca!([N; 2])` repeat expression and confirm it still errors; watch the tracking issues for when it lands.
3. Skim the project-const-generics repo / Zulip before filing duplicates — design feedback is solicited there.

Sources

- Generic Const Args and You — Inside Rust, 2026-10-02 (inspiration; see *Updated vs original* in the PR)
- generic_const_args — Unstable Book (tracking issue #151972)
- Full Const Generics — 2026 project goal
- Verified locally: `rustc 1.101.0-nightly (a30aa9064 2026-10-08)`

Summary

Summary

Chapter Key Takeaways

- Generics monomorphize at compile time for zero runtime overhead.
- Traits declare shared contracts across concrete types.
- Trait bounds enforce capabilities on generic type parameters.
- Trait objects (`dyn Trait`) provide dynamic dispatch using vtables when heterogeneous collections are needed.

Lifetimes: Complete Understanding

What Lifetimes Are (and Are Not)

What Lifetimes Are (and Are Not)

Every reference in Rust has a **lifetime**—the scope for which that reference is valid.

What Lifetimes Are Not

- Lifetimes do **not** change how long a value lives at runtime.
- Lifetimes do **not** allocate or free memory.
- Lifetimes are purely compile-time annotations that help the borrow checker ensure references never outlive the data they point to.

The Lifetime Elision Rules

The Lifetime Elision Rules

In common function patterns, Rust automatically infers lifetime parameters via three elision rules:

1. Each parameter that is a reference gets its own lifetime parameter (`fn foo<'a, 'b>(x: &'a i32, y: &'b i32)`).
2. If there is exactly one input lifetime parameter, that lifetime is assigned to all output lifetime parameters (`fn foo<'a>(x: &'a i32) -> &'a i32`).
3. If there are multiple input lifetime parameters, but one of them is `&self` or `&mut self`, the lifetime of `self` is assigned to all output lifetime parameters.

Annotating Lifetimes in Functions and Structs

Annotating Lifetimes in Functions and Structs

When a function returns a reference that could come from multiple inputs, explicit lifetime annotations are required:

```
// The returned reference is guaranteed valid for the intersection of 'a
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

struct Excerpt<'a> {
    part: &'a str,
}

fn main() {
    let string1 = String::from("desk configuration");
    let result;
    {
        let string2 = String::from("orders");
        result = longest(string1.as_str(), string2.as_str());
        println!("Longest string: {}", result);
    }
}
```

Lifetime Bounds on Traits

Lifetime Bounds on Traits

Constrain types with lifetime bounds, such as `T: 'a` (meaning `T` must outlive lifetime `'a`):

```
use std::fmt::Display;

fn print_ref<'a, T>(item: &'a T)
where
    T: Display + 'a,
{
    println!("Item: {}", item);
}
```

Advanced Lifetime Patterns

Advanced Lifetime Patterns

The `'static` Lifetime

A reference with `'static` lifetime can live for the entire duration of the program: - String literals: `&'static str` stored in the read-only data segment. - Owned values promoted to static duration.

Summary

Summary

Chapter Key Takeaways

- Lifetimes allow the compiler to prove reference validity at compile time.
- Elision rules save annotations in common single-reference or method signatures.
- Structs holding references must declare lifetime parameters (`Struct<'a>`).
- `'static` denotes references valid for the entire program runtime.

Smart Pointers and Interior Mutability

Box for Heap Allocation

Box for Heap Allocation

`Box<T>` allocates a value on the heap and provides a stack pointer with ownership semantics.

```
fn main() {
    let b = Box::new(42);
    println!("Box points to heap value: {}", *b);

    // Recursive data types require Box to have a known size:
    #[derive(Debug)]
    enum List {
        Cons(i32, Box<List>),
        Nil,
    }

    let list = List::Cons(1, Box::new(List::Cons(2, Box::new(List::Nil))));
    println!("Recursive list: {:?}", list);
}
```

Reference Counting with Rc and Arc

Reference Counting with Rc and Arc

When multiple parts of a program need shared read ownership of heap memory: - `Rc<T>`: Single-threaded reference counting. - `Arc<T>`: Atomic reference counting across multiple threads.

```
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3]);

    let data_clone = Arc::clone(&data);
    let handle = thread::spawn(move || {
        println!("Thread sees: {:?}", data_clone);
    });

    handle.join().unwrap();
    println!("Main sees: {:?}", data);
}
```

Interior Mutability with RefCell

Interior Mutability with RefCell

`RefCell<T>` enforces Rust's borrowing rules dynamically at runtime rather than at compile time:

```
use std::cell::RefCell;

fn main() {
    let val = RefCell::new(10);
    {
        let mut mut_borrow = val.borrow_mut();
        *mut_borrow += 5;
    }
    println!("Mutated value: {}", *val.borrow());
}
```

Cell and Mutex for Shared State

Cell and Mutex for Shared State

- `Cell<T>`: Interior mutability for `Copy` types via value copying.
- `Mutex<T>`: Thread-safe interior mutability via mutual exclusion locks.

```
use std::sync::Mutex;

fn main() {
    let m = Mutex::new(0);
    {
        let mut guard = m.lock().unwrap();
        *guard += 1;
    }
    println!("Mutex protected value: {:?}", m.lock().unwrap());
}
```

Weak References and Breaking Cycles

Weak References and Breaking Cycles

Cycles of `Rc` or `Arc` pointers can cause memory leaks. Use `Weak<T>` (downgraded non-owning reference) to break reference cycles:

```
use std::rc::{Rc, Weak};

struct Node {
    parent: Option<Weak<Node>>,
}
```

LazyLock and OnceLock

LazyLock and OnceLock

Global or shared values often need **one-time initialization**: a compiled regex, a config snapshot, a connection pool handle. Before Rust 1.70 / 1.80 you reached for `lazy_static`, `once_cell`, or a hand-rolled `Mutex<Option<T>>`. Today the standard library covers the boring cases:

Type	Module	Stabilized	Role
`OnceLock<T>`	`std::sync`	1.70	Thread-safe cell: write once, then shared reads
`LazyLock<T, F>`	`std::sync`	1.80	`OnceLock` + init closure; initializes on first access
`OnceCell<T>` / `LazyCell<T>`	`std::cell`	1.70 / 1.80	Single-threaded cousins (no `Sync`)

Prefer **`LazyLock`** for `static` values with a pure init function. Prefer **`OnceLock`** when initialization needs arguments, fallible setup, or happens explicitly in `main` / a builder. This chapter sits next to `Mutex` and `Arc` because the shared handle is often wrapped in those types later — but lazy init itself is not a lock around every read after the first.

Mental model

```
first access / set()

  run init exactly once      store T

 later access      &T (no re-init)
```

- Concurrent first accessors: one wins the init; others wait, then see the same `T`.
- If the init closure **panics**, `LazyLock` is **poisoned** for the rest of the process (unrecoverable). Fail fallible work *before* installing into a `LazyLock`, or use `OnceLock` + `Result` you control.
- `static LazyLock<T>` values are not dropped at process exit; the OS reclaims the memory. That is intentional and fine for process-lifetime data.

Worked examples

Case 1: `LazyLock` for a process-wide regex

```
use std::sync::LazyLock;
use regex::Regex;

static EMAIL: LazyLock<Regex> = LazyLock::new(|| {
    Regex::new(r"(?i)^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$").expect("static regex")
});

fn looks_like_email(s: &str) -> bool {
    EMAIL.is_match(s)
}

fn main() {
    assert!(looks_like_email("dev@example.com"));
    assert!(!looks_like_email("not-an-email"));
    println!("ok");
}
```

First call to `looks_like_email` compiles the pattern once. Later calls reuse the same `Regex`. The `expect` is acceptable only for patterns that are compile-time constants you control — never for user-supplied patterns inside a `LazyLock` init.

Add `regex` in `Cargo.toml` for this sample (`regex = "1"`). For a std-only sketch, replace the body with a pure function that builds a `String` or `HashMap`.

Case 2: `OnceLock` when init needs an argument

```
use std::sync::OnceLock;

struct AppConfig {
    api_root: String,
}

fn config() -> &'static AppConfig {
    static CFG: OnceLock<AppConfig> = OnceLock::new();
    CFG.get_or_init(|| AppConfig {
        api_root: std::env::var("API_ROOT").unwrap_or_else(|_| "http://127.0.0.1:8080".into(
    })
}

fn main() {
    println!("api_root={}", config().api_root);
    // Second call returns the same address
```

```
    assert!(std::ptr::eq(config(), config()));
}
```

`get_or_init` runs the closure only if empty. `set` / `get` let you initialize from `main` after CLI parsing:

```
use std::sync::OnceLock;

static CFG: OnceLock<String> = OnceLock::new();

fn main() {
    let root = std::env::args().nth(1).unwrap_or_else(|| "/srv/app".into());
    CFG.set(root).expect("config set once");
    println!("{}", CFG.get().expect("configured"));
}
```

`set` returns `Err(value)` if the cell was already filled — use that to catch double-init bugs in tests.

Case 3: Fallible init without poisoning `LazyLock`

```
use std::sync::OnceLock;

static DB_URL: OnceLock<String> = OnceLock::new();

fn init_db_url() -> Result<(), String> {
    let url = std::env::var("DATABASE_URL").map_err(|_| "DATABASE_URL missing".to_string())?
    if !url.starts_with("postgres://") {
        return Err("DATABASE_URL must start with postgres://".into());
    }
    DB_URL.set(url).map_err(|_| "DB_URL already set".to_string())?;
    Ok(())
}

fn db_url() -> &'static str {
    DB_URL.get().expect("call init_db_url() before use")
}

fn main() {
    if let Err(e) = init_db_url() {
        eprintln!("startup failed: {e}");
        std::process::exit(1);
    }
    println!("using {}", db_url());
}
```

Validate and then `set`. Do not hide I/O errors inside `LazyLock::new(|| …)` if you need a clean startup failure path.

Case 4: Single-threaded `LazyCell`

```rust
use std::cell::LazyCell;

fn main() {
    let cache = LazyCell::new(|| {
        println!("computing");
        21 * 2
    });
    assert_eq!(*cache, 42); // prints "computing"
    assert_eq!(*cache, 42); // silent
}
```

Use `LazyCell` / `OnceCell` inside one thread (or inside a type that is already `!Sync`). Crossing threads wants `LazyLock` / `OnceLock`.

Choosing among the options

Situation	Prefer
`static` value, infallible or `expect`-able init	`LazyLock`
Init needs CLI/env parsed in `main`	`OnceLock` + `set`
Fallible init with structured errors	`OnceLock` + `Result` before `set`
Non-`Sync` local cache	`LazyCell` / `OnceCell`
Mutate after init	Not these cells — use `Mutex` / `RwLock` around the data, or rebuild

`LazyLock` implements `Deref`, so `*LAZY` and method calls work like a plain `&T` after init. `OnceLock` exposes `get`, `get_or_init`, `set`, and related helpers explicitly.

Pitfalls

1. **Heavy work on first request path.** First HTTP request paying for a huge init surprises latency SLOs — warm `LazyLock` during startup (`let _ = &*STATIC;`) if the cost matters.
2. **Panic inside `LazyLock` init.** Poison is permanent for that static. Prefer fallible setup with `OnceLock`.
3. **Storing `Mutex` inside when you only needed one-time init.** After init, readers of an immutable `T` do not need a mutex. Add synchronization only for post-init mutation.
4. **Assuming `Drop` runs for `static LazyLock`.** It does not at process exit. Flush external resources explicitly if required.
5. **Using `once_cell` or `lazy_static` in new code on 1.80+.** Prefer std unless you need an API those crates still uniquely provide.

Try this

1. Build Case 1 (or a std-only `LazyLock<HashMap<…>>`) and print addresses of `&*EMAIL` across two threads with `std::thread::scope` — they must match.
2. Call `CFG.set` twice in Case 2's `set` variant; assert the second call is `Err`.
3. Replace Case 3's `OnceLock` with a `LazyLock` that `expect`s on missing env — then run without `DATABASE_URL` and observe the panic/poison behavior.
4. Skim `std::sync::LazyLock` and `OnceLock` for `get_mut` / `take` (where available on your MSRV) before using them in libraries.

Sources

- `std::sync::LazyLock` (1.80+)
- `std::sync::OnceLock` (1.70+)
- `std::cell::LazyCell` / `OnceCell`

MaybeUninit and Uninitialized Memory

MaybeUninit and Uninitialized Memory

Safe Rust assumes every value is **valid for its type**: a `bool` is only `true` or `false`, a reference is non-null and aligned, a `Vec<T>` upholds its length/capacity invariants. Sometimes you need storage **before** those invariants hold — out-pointers, element-by-element arrays, FFI buffers. The standard tool is `std::mem::MaybeUninit<T>`: a wrapper that tells the compiler "these bytes might not be initialized yet."

Do **not** reach for the deprecated `mem::uninitialized` or casual `mem::zeroed` on arbitrary `T`. Prefer `MaybeUninit`, initialize deliberately, then `assume_init` only when every validity requirement of `T` is met.

Mental model

```
MaybeUninit::uninit()     ← bytes may be garbage; Drop of T does NOT run

   write / ptr::write     ← initialize without dropping old (uninit) contents

 assume_init / assume_init_drop


       T                  ← normal Drop and safe APIs apply
```

API	Role
`MaybeUninit::uninit()`	Allocate storage; contents uninitialized
`MaybeUninit::new(v)`	Already-initialized wrapper
`MaybeUninit::zeroed()`	Fill with `0` bytes — **only** valid if all-zero is a valid `T`
`write(val)`	Initialize (overwrites without dropping previous)
`assume_init()`	Assert fully initialized; take ownership as `T`
`assume_init_drop()`	Drop in place when you do not want ownership
`as_mut_ptr()` / `&raw mut (*ptr).field`	Field-by-field init without creating `&mut` to uninit fields

Dropping a `MaybeUninit<T>` **never** runs `T`'s destructor. If you initialized and then abandon the wrapper without `assume_init` / `assume_init_drop`, you **leak** resources (safe but usually undesirable).

Worked examples

Case 1: `write` then `assume_init`

```
use std::mem::MaybeUninit;

fn main() {
    let mut slot = MaybeUninit::<Vec<u8>>::uninit();
    slot.write(vec![1, 2, 3]);
    let v = unsafe { slot.assume_init() };
    assert_eq!(v, vec![1, 2, 3]);
    println!("len={}", v.len());
}
```

`write` returns `&mut T` to the now-initialized contents. Calling `write` twice **skips** dropping the first value — a leak if `T` owns heap memory. Prefer one write, or drop explicitly between overwrites when you must replace.

Case 2: Out-pointer style

```
use std::mem::MaybeUninit;

unsafe fn fill_vec(out: *mut Vec<i32>) {
    // write does not drop old contents - correct for uninit destination
    unsafe { out.write(vec![10, 20, 30]); }
}

fn main() {
    let mut slot = MaybeUninit::uninit();
    unsafe { fill_vec(slot.as_mut_ptr()); }
    let v = unsafe { slot.assume_init() };
    assert_eq!(v, vec![10, 20, 30]);
}
```

The caller owns allocation strategy; the callee only initializes. After `assume_init`, `Drop` for `Vec` runs normally at end of scope.

Case 3: Array element-by-element

```
use std::mem::{self, MaybeUninit};

fn build_boxes() -> [Box<u32>; 4] {
    let mut data: [MaybeUninit<Box<u32>>; 4] = [const { MaybeUninit::uninit() }; 4];
```

```
    for (i, slot) in data.iter_mut().enumerate() {
        slot.write(Box::new(i as u32));
    }

    // SAFETY: every element was written
    unsafe { mem::transmute::<_, [Box<u32>; 4]>(data) }
}

fn main() {
    let arr = build_boxes();
    assert_eq!(*arr[3], 3);
    println!("{:?}", arr.iter().map(|b| **b).collect::<Vec<_>>());
}
```

Panic mid-loop leaks already-written `Box`es (no destructor on remaining `MaybeUninit`), but does not invent UB from dropping uninitialized boxes. For partial init, track a length and call `assume_init_drop` on the live prefix in a cleanup path.

Case 4: Field-by-field struct init

```
use std::mem::MaybeUninit;

#[derive(Debug, PartialEq)]
struct Person {
    name: String,
    age: u32,
}

fn main() {
    let person = {
        let mut uninit = MaybeUninit::<Person>::uninit();
        let ptr = uninit.as_mut_ptr();

        // SAFETY: write through raw field pointers - do NOT create &mut to uninit fields
        unsafe {
            (&raw mut (*ptr).name).write(String::from("Ada"));
            (&raw mut (*ptr).age).write(36);
            uninit.assume_init()
        }
    };

    assert_eq!(
        person,
        Person {
            name: String::from("Ada"),
            age: 36
```

```
        }
    );
    println!("{person:?}");
}
```

Using `assume_init_mut()` to "assign fields" **before** the struct is fully initialized is UB — that creates references to uninitialized memory. Prefer `&raw mut` / `ptr::write` as above. Padding bytes need not be initialized.

Case 5: `zeroed` pitfalls

```
use std::mem::MaybeUninit;

fn main() {
    // OK: (u8, bool) can be all-zero
    let pair = unsafe { MaybeUninit::<(u8, bool)>::zeroed().assume_init() };
    assert_eq!(pair, (0, false));

    // NOT OK: &T must be non-null - zeroed reference is instantaneous UB
    // let bad: &i32 = unsafe { MaybeUninit::zeroed().assume_init() };

    // NOT OK: many enums reject discriminant 0
    // enum NonZero { A = 1, B = 2 }
    // let bad = unsafe { MaybeUninit::<NonZero>::zeroed().assume_init() };

    println!("pair={pair:?}");
}
```

Rule of thumb: `zeroed().assume_init()` only for types where **every bit pattern of all-zeros is a valid value** (raw integers, `bool`, structs of such). Never for references, `NonNull`, most `enums`, or "looks like a Vec with ptr=0".

Choosing the API

Situation	Prefer
Single value, known initializer	`MaybeUninit::new` or plain `T`
Deferred init / out-pointer	`uninit` + `write` / `ptr::write`
Large array filled in a loop	`[MaybeUninit<T>; N]` + `write` each
Struct fields in sequence	`&raw mut (*ptr).field` + `write`
Need all-zero bit pattern and it is valid	`zeroed` with care
Old code with `mem::uninitialized`	Port to `MaybeUninit`

Pitfalls

1. **`assume_init` too early** — immediate UB, even for `i32` / `bool` when bytes are still uninit.
2. **`zeroed` on references / invalid enums** — classic footgun; all-zero is not "safe default" for every `T`.
3. **Double `write` without drop** — leaks the first `T`.
4. **Creating `&` / `&mut` to uninitialized fields** — use raw pointers / `&raw mut` until fully init.
5. **Transmuting `&mut T` to `&mut MaybeUninit<T>` for safe code** — lets safe code invent uninit views; unsound.
6. **Assuming Miri-silence equals production-proof** — still run tests under Miri for unsafe modules when you can.

Try this

1. Run Case 1 and Case 3 under `cargo +nightly miri run` if Miri is available.
2. Deliberately comment out one field write in Case 4 and observe that Miri / careful review catches the incomplete init (do not ship it).
3. Replace a `mem::uninitialized` call in an old snippet with `MaybeUninit` and list which `assume_init` proof you now owe.
4. Skim the Rustonomicon chapter on unchecked uninitialized memory and compare its array pattern to Case 3.

Sources

- `std::mem::MaybeUninit`
- The Rustonomicon — Unchecked Uninitialized Memory

Custom Allocators and the Allocator API

Custom Allocators and the Allocator API

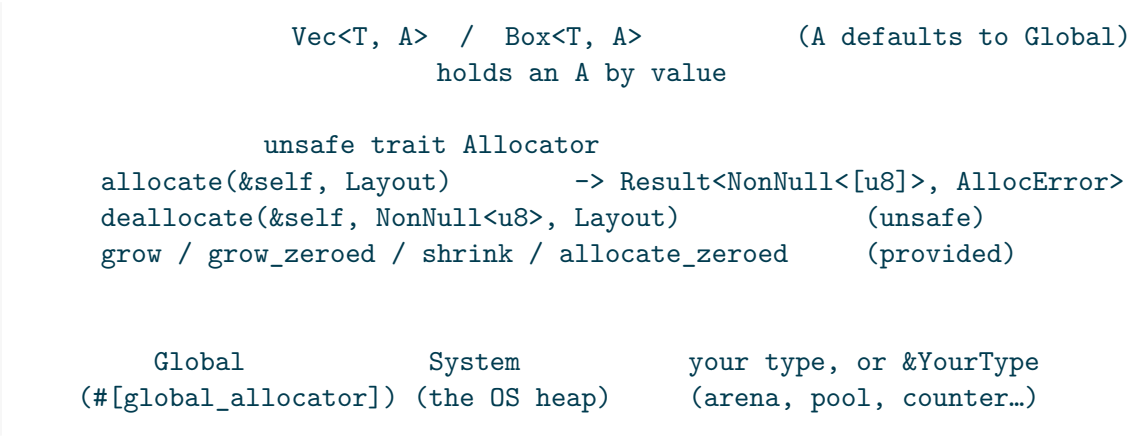
Thesis

Every `Box` and `Vec` you have written so far got its memory from one place: the **global allocator**. You could swap that allocator for the whole program with `#[global_allocator]`, but you could not say "*this* vector lives in *that* arena". The `Allocator` trait that would let you do so has existed since 2016, behind the nightly-only `allocator_api` feature, and arena crates have shipped nightly-only allocator-aware APIs for years because of it.

Rust 1.100 (scheduled for 2026-11-12) stabilizes a deliberately small slice of that design: the `std::alloc::Allocator` trait, the `Global` and `System` allocators as implementors, and allocator-aware constructors on exactly two containers, `Box<T, A>` and `Vec<T, A>`. Everything else (`Rc::new_in`, fallible `try_*_in` constructors, allocator-aware `String` and `HashMap`) stays unstable under a new feature name, `allocator_ext`.

This chapter builds two allocators on the stable surface, a counting wrapper and a bump arena. It then walks through the places where the type system and the abort-on-failure policy catch people out. Everything was compiled with the current beta, which is what becomes 1.100.

Mental model



Four facts carry the chapter:

Fact	Consequence
The allocator is a **type parameter** with a default: `Vec<T, A = Global>`	`Vec<u32>` and `Vec<u32, System>` are *different types*. Code written for one does not accept the other

Fact	Consequence
The container **stores** an `A`	A zero-sized allocator (`Global`, `System`) costs nothing; `&Arena` costs one pointer; `&dyn Allocator` costs two
Methods take `&self`	Implement `Allocator` on `&Arena` (blanket impl provided) to share one arena among many containers. Interior mutability (`Cell`) does the bookkeeping
The trait is `unsafe` to implement	You promise that returned blocks are valid until deallocated, that blocks don't overlap, and that no method unwinds. The compiler cannot check any of it

`allocate` returns `NonNull<[u8]>`, a *slice* pointer, so an allocator can report that it handed back more than was asked for. `AllocError` is a zero-sized "no" with no further detail.

Worked examples

All listings were compiled and run on the box with **rustc 1.100.0-beta.4 (2026-10-04)** and, for comparison, **rustc 1.99.0** stable, both installed with `rustup`. Until 1.100 reaches stable, use `+beta`:

```
rustup toolchain install beta --profile minimal
rustc +beta --version
```

```
rustc 1.100.0-beta.4 (a2eb6097d 2026-10-04)
```

Case 1: a counting allocator and a bump arena

```
cargo +beta new alloc-demo && cd alloc-demo
```

`Counting` wraps any other allocator and records each call. `Arena<N>` hands out slices of a fixed buffer by bumping a cursor, and its `deallocate` does nothing: the whole region is released at once when the arena is dropped. Note that neither needs a `#![feature]` line.

```
// src/main.rs - a counting allocator and a bump arena on the stable Allocator API (Rust 1.1
use std::alloc::{AllocError, Allocator, Global, Layout};
use std::cell::{Cell, UnsafeCell};
use std::mem::MaybeUninit;
use std::ptr::NonNull;

/// Wraps another allocator and counts what passes through it.
struct Counting<A: Allocator = Global> {
    inner: A,
    allocs: Cell<usize>,
    grows: Cell<usize>,
```

```
    frees: Cell<usize>,
    live_bytes: Cell<usize>,
}

impl Counting {
    const fn new() -> Self {
        Counting { inner: Global, allocs: Cell::new(0), grows: Cell::new(0), frees: Cell::ne
    }
    fn report(&self, label: &str) {
        println!(
            "{label:<22} allocs={} grows={} frees={} live_bytes={}",
            self.allocs.get(), self.grows.get(), self.frees.get(), self.live_bytes.get()
        );
    }
}

// SAFETY: every method forwards to `inner` with the caller's arguments unchanged,
// so blocks stay valid until deallocated and are freed by the allocator that made them.
unsafe impl<A: Allocator> Allocator for Counting<A> {
    fn allocate(&self, layout: Layout) -> Result<NonNull<[u8]>, AllocError> {
        self.allocs.set(self.allocs.get() + 1);
        self.live_bytes.set(self.live_bytes.get() + layout.size());
        self.inner.allocate(layout)
    }
    unsafe fn deallocate(&self, ptr: NonNull<u8>, layout: Layout) {
        self.frees.set(self.frees.get() + 1);
        self.live_bytes.set(self.live_bytes.get() - layout.size());
        unsafe { self.inner.deallocate(ptr, layout) }
    }
    unsafe fn grow(&self, ptr: NonNull<u8>, old: Layout, new: Layout) -> Result<NonNull<[u8]
        self.grows.set(self.grows.get() + 1);
        self.live_bytes.set(self.live_bytes.get() - old.size() + new.size());
        unsafe { self.inner.grow(ptr, old, new) }
    }
}

/// A fixed-size bump arena: allocation moves a cursor, deallocation does nothing,
/// and all memory comes back at once when the arena itself is dropped.
struct Arena<const N: usize> {
    buf: UnsafeCell<[MaybeUninit<u8>; N]>,
    used: Cell<usize>,
}

impl<const N: usize> Arena<N> {
    fn new() -> Self {
        Arena { buf: UnsafeCell::new([MaybeUninit::uninit(); N]), used: Cell::new(0) }
    }
```

```
    fn used(&self) -> usize {
        self.used.get()
    }
}

// SAFETY: blocks are carved from `buf` without overlap and stay valid while `self`
// is alive; borrowing the arena (`&Arena`) ties every collection to that lifetime.
unsafe impl<const N: usize> Allocator for Arena<N> {
    fn allocate(&self, layout: Layout) -> Result<NonNull<[u8]>, AllocError> {
        let base = self.buf.get() as *mut u8;
        let start = self.used.get();
        let pad = base.wrapping_add(start).align_offset(layout.align());
        let begin = start.checked_add(pad).ok_or(AllocError)?;
        let end = begin.checked_add(layout.size()).ok_or(AllocError)?;
        if end > N {
            return Err(AllocError);
        }
        self.used.set(end);
        let ptr = unsafe { NonNull::new_unchecked(base.add(begin)) };
        Ok(NonNull::slice_from_raw_parts(ptr, layout.size()))
    }
    unsafe fn deallocate(&self, _ptr: NonNull<u8>, _layout: Layout) {
        // Bump arenas free everything at once, when the arena is dropped.
    }
}

#[derive(Debug)]
struct Token<'a> {
    kind: &'static str,
    text: &'a str,
}

fn tokenize<'s, A: Allocator>(src: &'s str, alloc: A) -> Vec<Token<'s>, A> {
    let mut out = Vec::new_in(alloc);
    for word in src.split_whitespace() {
        let kind = if word.chars().all(|c| c.is_ascii_digit()) { "num" } else { "word" };
        out.push(Token { kind, text: word });
    }
    out
}

fn main() {
    // 1. Counting: watch Vec's growth policy through the allocator.
    let counter = Counting::new();
    {
        let mut v: Vec<u64, &Counting> = Vec::new_in(&counter);
        counter.report("after new_in");
```

```
        for i in 0..100 {
            v.push(i);
        }
        counter.report("after 100 pushes");
        let b = Box::new_in([0u8; 256], &counter);
        counter.report("plus a Box");
        println!("capacity = {}, box len = {}", v.capacity(), b.len());
    }
    counter.report("after drop");

    let counter2 = Counting::new();
    {
        let mut v: Vec<u64, &Counting> = Vec::with_capacity_in(100, &counter2);
        for i in 0..100 {
            v.push(i);
        }
    }
    counter2.report("with_capacity_in(100)");

    // 2. Arena: many small values, one region, no per-object free.
    let arena: Arena<4096> = Arena::new();
    let toks = tokenize("let x = 42 + y * 7", &arena);
    println!("{} tokens, arena used = {} bytes", toks.len(), arena.used());
    println!("first = {:?}, last = {:?}", toks[0], toks[toks.len() - 1]);
    let nums: Vec<&str> = toks.iter().filter(|t| t.kind == "num").map(|t| t.text).collect();
    println!("numbers = {nums:?}");

    let pair = Box::new_in((1u32, 2u32), &arena);
    println!("boxed pair = {:?}, arena used = {}", *pair, arena.used());
    println!("Vec allocator is the arena: {}", std::ptr::eq(*toks.allocator(), &arena));
}
```

```
cargo +beta run -q
```

```
after new_in           allocs=0 grows=0 frees=0 live_bytes=0
after 100 pushes       allocs=1 grows=5 frees=0 live_bytes=1024
plus a Box             allocs=2 grows=5 frees=0 live_bytes=1280
capacity = 128, box len = 256
after drop             allocs=2 grows=5 frees=2 live_bytes=0
with_capacity_in(100)  allocs=1 grows=0 frees=1 live_bytes=0
8 tokens, arena used = 384 bytes
first = Token { kind: "word", text: "let" }, last = Token { kind: "num", text: "7" }
numbers = ["42", "7"]
boxed pair = (1, 2), arena used = 392
Vec allocator is the arena: true
```

What to notice:

- **`Vec::new_in` allocates nothing.** The first `push` makes one allocation (4 × `u64`), and then the vector *grows* five times (8, 16, 32, 64, 128) to hold 100 items. `with_capacity_in(100)` makes one allocation and no growth. The counter turns a growth policy you normally take on faith into numbers you can see.
- **`live_bytes` returns to 0** after the scope ends, so every byte allocated was deallocated with the layout it was allocated with. That is the contract `Vec` and `Box` keep on their side.
- **The arena used 384 bytes for 8 tokens of 32 bytes each.** The provided `grow` method allocates a new, larger block, copies, and calls `deallocate` on the old block. In a bump arena that call is a no-op, so the 4-token block (128 bytes) is simply abandoned. Arenas reward `with_capacity_in` far more than the heap does.
- `tokenize` is generic over `A: Allocator` and returns `Vec<Token<'s>, A>`. The caller, not the function, decides where the tokens live.
- `toks.allocator()` gives back the allocator the vector holds. Here that is the `&Arena` reference.

On 1.99 stable the same crate is rejected at the first import:

```
cargo +stable build -q
```

```
error[E0658]: use of unstable library feature `allocator_api`
 --> src/main.rs:2:18
  |
2 | use std::alloc::{AllocError, Allocator, Global, Layout};
  |                  ^^^^^^^^^^
  |
  = note: see issue #32838 <https://github.com/rust-lang/rust/issues/32838> for more informa
...
```

Case 2: what the allocator parameter costs

The allocator is stored inside the container, so its size shows up in the container's size. The trait is also `dyn`-compatible, so you can choose an allocator at run time.

```
// dynalloc.rs - choosing an allocator at run time with &dyn Allocator
use std::alloc::{Allocator, Global, System};

fn pick(use_system: bool) -> &'static dyn Allocator {
    if use_system { &System } else { &Global }
}

fn main() {
    for flag in [false, true] {
        let alloc = pick(flag);
        let mut v: Vec<u32, &dyn Allocator> = Vec::with_capacity_in(4, alloc);
        v.extend([10, 20, 30]);
        let sum: u32 = v.iter().sum();
```

```
        println!("system={flag} sum={sum} size_of::<Vec<u32, &dyn Allocator>>()={}",
                 std::mem::size_of_val(&v));
    }
    println!("size_of::<Vec<u32>>() = {}", std::mem::size_of::<Vec<u32>>());
    println!("size_of::<Vec<u32, System>>() = {}", std::mem::size_of::<Vec<u32, System>>());
}
```

```
rustc +beta --edition 2024 dynalloc.rs && ./dynalloc
```

```
system=false sum=60 size_of::<Vec<u32, &dyn Allocator>>()=40
system=true sum=60 size_of::<Vec<u32, &dyn Allocator>>()=40
size_of::<Vec<u32>>() = 24
size_of::<Vec<u32, System>>() = 24
```

`Global` and `System` are zero-sized, so a `Vec` that uses them is still three words. A `&Arena` adds one word, and a `&dyn Allocator` adds two (data pointer and vtable) plus an indirect call on every allocation. Reach for `dyn` only when the choice really is made at run time.

Case 3: an allocator says no

Every allocating method you already know (`push`, `extend`, `Box::new_in`) is **infallible**. When the allocator returns `AllocError`, they call `handle_alloc_error`, which aborts the process. The global heap almost never says no, so this hardly matters there. A 256-byte arena says no quickly.

```
// exhaust.rs - what happens when a custom allocator says no
use std::alloc::{AllocError, Allocator, Layout};
use std::cell::{Cell, UnsafeCell};
use std::mem::MaybeUninit;
use std::ptr::NonNull;

struct Arena<const N: usize> {
    buf: UnsafeCell<[MaybeUninit<u8>; N]>,
    used: Cell<usize>,
}

impl<const N: usize> Arena<N> {
    fn new() -> Self {
        Arena { buf: UnsafeCell::new([MaybeUninit::uninit(); N]), used: Cell::new(0) }
    }
}

// SAFETY: non-overlapping blocks carved from `buf`, valid while the arena lives.
unsafe impl<const N: usize> Allocator for Arena<N> {
    fn allocate(&self, layout: Layout) -> Result<NonNull<[u8]>, AllocError> {
        let base = self.buf.get() as *mut u8;
```

```
        let start = self.used.get();
        let pad = base.wrapping_add(start).align_offset(layout.align());
        let begin = start.checked_add(pad).ok_or(AllocError)?;
        let end = begin.checked_add(layout.size()).ok_or(AllocError)?;
        if end > N {
            return Err(AllocError);
        }
        self.used.set(end);
        let ptr = unsafe { NonNull::new_unchecked(base.add(begin)) };
        Ok(NonNull::slice_from_raw_parts(ptr, layout.size()))
    }
    unsafe fn deallocate(&self, _: NonNull<u8>, _: Layout) {}
}

fn main() {
    let mode = std::env::args().nth(1).unwrap_or_default();
    let arena: Arena<256> = Arena::new();
    let mut v: Vec<u64, &Arena<256>> = Vec::new_in(&arena);

    if mode == "try" {
        for i in 0.. {
            if v.len() == v.capacity() {
                if let Err(e) = v.try_reserve(1) {
                    println!("stopped at len={} cap={}: {e}", v.len(), v.capacity());
                    break;
                }
            }
            v.push(i);
        }
        println!("arena used = {} of 256", arena.used.get());
    } else {
        for i in 0..64u64 {
            v.push(i); // infallible API: allocation failure aborts the process
            println!("len={} cap={} used={}", v.len(), v.capacity(), arena.used.get());
        }
    }
}
```

```
rustc +beta --edition 2024 -O exhaust.rs
./exhaust; echo "exit=$?"
./exhaust try; echo "exit=$?"
```

```
len=1 cap=4 used=32
len=2 cap=4 used=32
len=3 cap=4 used=32
len=4 cap=4 used=32
```

```
len=5 cap=8 used=96
len=6 cap=8 used=96
len=7 cap=8 used=96
len=8 cap=8 used=96
len=9 cap=16 used=224
len=10 cap=16 used=224
len=11 cap=16 used=224
len=12 cap=16 used=224
len=13 cap=16 used=224
len=14 cap=16 used=224
len=15 cap=16 used=224
len=16 cap=16 used=224
memory allocation of 256 bytes failed
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
Aborted
exit=134
stopped at len=16 cap=16: memory allocation failed because the memory allocator returned an
arena used = 224 of 256
exit=0
```

(`Aborted` is the shell reporting `SIGABRT`; its exact wording differs between shells.) The abort is not a panic. `catch_unwind` cannot stop it, and destructors do not run. `Vec::try_reserve` *is* available on `Vec<T, A>` (it has been stable since 1.57 and works with any allocator), so the fallible path is to reserve before you push and handle the `TryReserveError`. Note too that 128 bytes of real data cost 224 bytes of arena, because the 32- and 64-byte blocks left behind by each growth step are never reused.

Case 4: what is *not* stable yet

The stabilization is an MVP on purpose. Three things people reach for straight away still fail on 1.100 beta:

```
// not_yet.rs - allocator-aware APIs that are still unstable in Rust 1.100
use std::alloc::{AllocError, Allocator, Global, Layout};
use std::ptr::NonNull;
use std::rc::Rc;

struct Pool;
unsafe impl Allocator for Pool {
    fn allocate(&self, l: Layout) -> Result<NonNull<[u8]>, AllocError> { Global.allocate(l)
    unsafe fn deallocate(&self, p: NonNull<u8>, l: Layout) { unsafe { Global.deallocate(p, l
}

fn main() {
    let r = Rc::new_in(5u8, Pool);                              // shared ownership in a cus
    let v = Vec::<u8, Pool>::try_with_capacity_in(8, Pool);     // fallible constructor
```

```
    let p = Box::into_pin(Box::new_in(1u8, Pool));              // pinning a custom-allocate
    println!("{r} {:?} {p}", v.map(|v| v.len()));
}
```

```
rustc +beta --edition 2024 not_yet.rs
```

```
error[E0658]: use of unstable library feature `allocator_ext`
  --> not_yet.rs:13:13
   |
13 |     let r = Rc::new_in(5u8, Pool);                                 // shared ownership in
   |             ^^^^^^^^^^
   |
   = note: see issue #163177 <https://github.com/rust-lang/rust/issues/163177> for more info

error[E0658]: use of unstable library feature `allocator_ext`
  --> not_yet.rs:14:13
   |
14 |     let v = Vec::<u8, Pool>::try_with_capacity_in(8, Pool);     // fallible constructor
   |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: see issue #163177 <https://github.com/rust-lang/rust/issues/163177> for more info

error[E0277]: the trait bound `Pool: StaticAllocator` is not satisfied
  --> not_yet.rs:15:27
   |
15 |     let p = Box::into_pin(Box::new_in(1u8, Pool));              // pinning a custom-all
   |             ------------- ^^^^^^^^^^^^^^^^^^^^^^ unsatisfied trait bound
   |             |
   |             required by a bound introduced by this call
   |
help: the nightly-only, unstable trait `StaticAllocator` is not implemented for `Pool`
...
```

- **`Rc::new_in` / `Arc::new_in`** remain under `allocator_ext` (tracking issue #163177). What *is* stable is `impl Allocator for Rc<A>` / `Arc<A>`, so a shared allocator can be passed around by `Rc`, but the reference-counted *values* cannot live in a custom allocator yet.
- **`try_with_capacity_in`** and the other fallible constructors are unstable. Use `new_in` plus `try_reserve`, as in Case 3.
- **`Box::into_pin`** needs the unstable `StaticAllocator` marker. `Global`, `System`, `&A` and `&mut A` have it, and your own types cannot opt in on stable. This is a soundness fix from the stabilization work: a box whose allocator can be dropped early must not promise `Pin`'s "never moves, never freed without drop" guarantee.
- **`String`, `HashMap`, `BTreeMap`, `VecDeque`** have no allocator parameter on stable. `String<System>` is `error[E0107]: struct takes 0 generic arguments`. If you need allocator-aware text, hold a `Vec<u8, A>` and validate it with `std::str::from_utf8`.

The trap

A collection outlives its arena

`&Arena` implements `Allocator`, and the borrow checker treats it like any other borrow. Returning a vector that borrows a local allocator fails to compile:

```
// outlive.rs - a collection cannot outlive the arena it borrows
use std::alloc::{AllocError, Allocator, Global, Layout};
use std::ptr::NonNull;

struct Pool; // stand-in: forwards to Global
unsafe impl Allocator for Pool {
    fn allocate(&self, l: Layout) -> Result<NonNull<[u8]>, AllocError> { Global.allocate(l)
    unsafe fn deallocate(&self, p: NonNull<u8>, l: Layout) { unsafe { Global.deallocate(p, l
}

fn build() -> Vec<u32, &'static Pool> {
    let pool = Pool;
    let mut v = Vec::new_in(&pool);
    v.push(1);
    v
}

fn main() {
    println!("{:?}", build());
}
```

```
rustc +beta --edition 2024 outlive.rs
```

```
error[E0515]: cannot return value referencing local variable `pool`
  --> outlive.rs:15:5
   |
13 |     let mut v = Vec::new_in(&pool);
   |                              ----- `pool` is borrowed here
14 |     v.push(1);
15 |     v
   |     ^ returns a value referencing data owned by the current function

error: aborting due to 1 previous error
```

This error is the feature working. It is the reason an arena in Rust can be safe without a garbage collector. The fix is structural: create the arena in the caller and pass `&arena` down, as `tokenize` does in Case 1.

`Vec<T, A>` does not fit where `Vec<T>` is expected

```
// mismatch.rs - Vec<T, A> is a different type from Vec<T>
use std::alloc::System;

fn total(v: &Vec<u32>) -> u32 {
    v.iter().sum()
}

fn main() {
    let mut v = Vec::new_in(System);
    v.extend([1, 2, 3]);
    println!("{}", total(&v));
}
```

```
error[E0308]: mismatched types
  --> mismatch.rs:11:26
   |
11 |     println!("{}", total(&v));
   |                     ----- ^^ expected `&Vec<u32>`, found `&Vec<{integer}, System>`
   |                     |
   |                     arguments to this function are incorrect
   |
   = note: expected reference `&Vec<u32, std::alloc::Global>`
              found reference `&Vec<{integer}, System>`
```

Every API in your codebase that names `&Vec<T>` or `Box<T>` quietly means `Global`. Read-only functions should take `&[T]`, which every `Vec<T, A>` derefs to. Functions that grow the vector should be generic over the allocator:

```
// mismatch_fixed.rs - accept slices, or be generic over the allocator
use std::alloc::{Allocator, System};

fn total(v: &[u32]) -> u32 {
    v.iter().sum()
}

fn push_twice<A: Allocator>(v: &mut Vec<u32, A>, x: u32) {
    v.push(x);
    v.push(x);
}

fn main() {
    let mut v = Vec::new_in(System);
    v.extend([1, 2, 3]);
    push_twice(&mut v, 10);
```

```
    println!("{}", total(&v));
    let plain = vec![4u32, 5];
    println!("{}", total(&plain));
}
```

```
26
9
```

Implementing the trait is a safety contract

`unsafe impl Allocator` is a promise the compiler cannot check. The ways to break it are the classic C allocator bugs: handing out overlapping blocks (a bump cursor that forgets alignment padding), returning memory that dies before the container does (an allocator that owns its buffer by value and gets *moved*), or panicking inside `allocate`. The stabilized contract explicitly forbids unwinding out of any `Allocator` method, from its clones, or from its drop. Keep the `unsafe impl` small, put the bookkeeping in plain safe code, and test it under Miri (`cargo +nightly miri test`), which catches the use-after-free, out-of-bounds, and aliasing violations that a broken allocator causes in the containers using it.

Choosing an approach

Need	Use
Replace the heap for the whole program	`#[global_allocator]` with a `GlobalAlloc` (stable since 1.28)
Many short-lived objects freed together (parsers, per-request scratch)	A bump arena passed as `&Arena` to `Vec::new_in` / `Box::new_in`
Measure or limit what one subsystem allocates	A wrapper like `Counting<A>` around `Global`
Pick the allocator at run time	`&dyn Allocator` (two extra words per container)
Shared ownership, maps, strings in a custom allocator	Not on stable yet (`allocator_ext`), so keep a third-party collection for now
MSRV below 1.100	Keep using nightly-gated code or the arena crate's own types; don't half-adopt

The boring rule

Pass the allocator in, don't hide it: `fn f<A: Allocator>(…, alloc: A) -> Vec<T, A>`. Take `&[T]` when you only read. Reserve capacity up front in arenas, and use `try_reserve` wherever running out is a normal outcome rather than a bug. Write `unsafe impl Allocator` only after you've measured a reason, and run it under Miri.

Try this

1. Change `Counting` to also override `shrink`, call `v.shrink_to_fit()` after the 100 pushes, and confirm the new counter moves while `live_bytes` drops to 800.
2. Give `Arena` a `reset(&mut self)` that sets `used` back to 0. Why does it have to take `&mut self`, and what does the borrow checker say if a `Vec<_, &Arena>` is still alive when you call it?
3. Make `Arena::deallocate` reclaim space when the block being freed is the most recent one (`ptr + size == cursor`). Re-run Case 1 and compare `arena used` before and after.
4. Rewrite `tokenize` to call `Vec::with_capacity_in(src.split_whitespace().count(), alloc)` and predict the new `arena used` value before running it.
5. Wrap `System` in `Counting<System>` and verify that `Box::into_pin` fails to compile for it even though it compiles for plain `System`. Which unstable trait is missing, and why can't you implement it yourself?

Sources

- rust-lang/rust PR #156882, "alloc: stabilise `Allocator`" (stabilization report, merged 2026-09-23, milestone 1.100.0): https://github.com/rust-lang/rust/pull/156882
- Draft release notes for 1.100.0: https://github.com/rust-lang/rust/issues/163811
- Tracking issue `allocator_api` #32838 and the remaining-surface issue `allocator_ext` #163177
- RFC 1398, "Kinds of allocators": https://rust-lang.github.io/rfcs/1398-kinds-of-allocators.html
- std docs: `std::alloc::Allocator`, `Vec::new_in`, `Box::new_in`, `Vec::try_reserve`, `handle_alloc_error`

Summary

Summary

Chapter Key Takeaways

- `Box<T>` allocates on the heap with single ownership.
- `Rc<T>` and `Arc<T>` provide shared read ownership via reference counting.
- `RefCell<T>` provides interior mutability with runtime borrow checks.
- `Mutex<T>` and `RwLock<T>` provide thread-safe interior mutability.
- `Weak<T>` prevents reference count cycles.
- `OnceLock` / `LazyLock` (and `OnceCell` / `LazyCell`) cover one-time initialization without third-party crates.
- `MaybeUninit<T>` allows working with uninitialized memory safely before type invariants are established.

Iterators, Closures, and Functional Patterns

The Iterator Trait and Protocol

The Iterator Trait and Protocol

In Rust, iterators are lazy sequences that produce values on demand via the `Iterator` trait:

```
fn main() {
    let v = vec![1, 2, 3];
    let mut iter = v.iter();

    assert_eq!(iter.next(), Some(&1));
    assert_eq!(iter.next(), Some(&2));
    assert_eq!(iter.next(), Some(&3));
    assert_eq!(iter.next(), None);
}
```

Creating Custom Iterators

Creating Custom Iterators

```
struct Counter {
    count: usize,
}

impl Counter {
    fn new() -> Self { Self { count: 0 } }
}

impl Iterator for Counter {
    type Item = usize;
    fn next(&mut self) -> Option<Self::Item> {
        if self.count < 3 {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

fn main() {
    let counter = Counter::new();
    let items: Vec<usize> = counter.collect();
    println!("Custom iterator output: {:?}", items);
}
```

Closures and Capture Semantics

Closures and Capture Semantics

Closures are anonymous functions that can capture environment variables: - `Fn`: Borrows values from the environment immutably. - `FnMut`: Borrows values mutably. - `FnOnce`: Takes ownership and consumes captured values.

```
fn main() {
    let factor = 3;
    let multiply = |x| x * factor;
    println!("Result: {}", multiply(10));
}
```

Higher-Order Functions and Combinators

Higher-Order Functions and Combinators

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5, 6];

    let sum_of_even_squares: i32 = numbers
        .iter()
        .filter(|&&x| x % 2 == 0)
        .map(|&x| x * x)
        .sum();

    println!("Sum of even squares: {}", sum_of_even_squares);
}
```

Lazy Evaluation and Performance

Lazy Evaluation and Performance

Rust iterator adaptors are **zero-cost abstractions**. They compile down to unrolled assembly with no heap allocation, often outperforming hand-written imperative loops.

Summary

Summary

Chapter Key Takeaways

- Iterators are lazy; evaluation only happens when consumed (e.g., `collect`, `sum`).
- Closures capture environment variables via `Fn`, `FnMut`, or `FnOnce`.
- Combinators (`map`, `filter`, `fold`) form clear functional pipelines.
- Modern Rust compilers optimize iterator pipelines into highly vectorized machine loops.

Concurrency: Fearless Parallelism

Spawning Threads with std::thread

Spawning Threads with std::thread

Rust guarantees thread safety through its type system.

```
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        println!("Worker thread running");
    });

    handle.join().unwrap();
    println!("Worker thread joined");
}
```

Message Passing with Channels

Message Passing with Channels

"Do not communicate by sharing memory; instead, share memory by communicating."

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let msg = String::from("order_payload");
        tx.send(msg).unwrap(); // Ownership moves into channel!
    });

    let received = rx.recv().unwrap();
    println!("Received through channel: {}", received);
}
```

Shared State with Mutex and RwLock

Shared State with Mutex and RwLock

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..5 {
        let counter_clone = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            let mut num = counter_clone.lock().unwrap();
            *num += 1;
        }));
    }

    for h in handles { h.join().unwrap(); }
    println!("Final count: {}", *counter.lock().unwrap());
}
```

Atomic Operations for Lock-Free Code

Atomic Operations for Lock-Free Code

For primitive integer updates, atomic instructions offer ultra-low latency lock-free synchronization:

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::Arc;
use std::thread;

fn main() {
    let atomic_val = Arc::new(AtomicUsize::new(0));
    let mut handles = vec![];

    for _ in 0..4 {
        let val_clone = Arc::clone(&atomic_val);
        handles.push(thread::spawn(move || {
            val_clone.fetch_add(1, Ordering::SeqCst);
        }));
    }

    for h in handles { h.join().unwrap(); }
    println!("Atomic total: {}", atomic_val.load(Ordering::SeqCst));
}
```

Send and Sync Traits Explained

Send and Sync Traits Explained

Rust's type system encodes concurrency rules into two marker traits: - `Send`: Indicates ownership of the type can be transferred across thread boundaries. - `Sync`: Indicates it is safe to share references to the type across multiple threads (`T: Sync` implies `&T: Send`).

Summary

Summary

Chapter Key Takeaways

- Threads are spawned via `std::thread::spawn` with ownership transfer via `move`.
- Channels enable safe message passing without shared locks.
- `Arc<Mutex<T>>` provides safe shared mutable state across threads.
- `Send` and `Sync` automatically prevent data races at compile time.

Project 3: Parallel Data Processing Pipeline with rayon

Parallel Data Processing Pipeline with rayon

Parallel Data Processing Pipeline with rayon

In Project 3, we implement a high-throughput parallel text analysis pipeline using `rayon`.

```
// Parallel pipeline conceptual architecture
pub struct TextStats {
    pub word_count: usize,
    pub line_count: usize,
}

pub fn analyze_lines(lines: &[&str]) -> TextStats {
    // rayon would process using .par_iter()
    let line_count = lines.len();
    let word_count = lines.iter().map(|l| l.split_whitespace().count()).sum();
    TextStats { word_count, line_count }
}

fn main() {
    let sample = vec![
        "Rust delivers safe systems programming",
        "Rayon provides work-stealing parallelism",
        "Zero-cost abstractions minimize runtime overhead",
    ];
    let stats = analyze_lines(&sample);
    println!("Processed {} lines, {} words", stats.line_count, stats.word_count);
}
```

Testing the Text Analyzer

Testing the Text Analyzer

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_text_analysis() {
        let input = vec!["one two three", "four five"];
        let stats = analyze_lines(&input);
        assert_eq!(stats.line_count, 2);
        assert_eq!(stats.word_count, 5);
    }
}
```

Benchmarking the Text Analyzer

Benchmarking the Text Analyzer

Comparing sequential iteration vs. Rayon parallel iteration across multi-megabyte corpora shows linear scaling up to physical core count.

Deploying the Text Analyzer

Deploying the Text Analyzer

Package the parallel text analyzer as a standalone CLI utility:

```
cargo build --release
./target/release/text_analyzer --input large_corpus.txt
```

Asynchronous Programming with async/await

The Future Trait and Async Basics

The Future Trait and Async Basics

Asynchronous Rust uses cooperative multitasking where tasks yield execution rather than blocking threads:

```
// The core Future trait in std
// pub trait Future {
//     type Output;
//     fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
// }

async fn compute_async_val() -> u32 {
    42
}
```

Running Async Code with Tokio

Running Async Code with Tokio

Async code requires an asynchronous executor runtime. Tokio is the de facto industry standard:

```
// Using tokio::main macro
// #[tokio::main]
// async fn main() {
//     let result = fetch_remote().await;
//     println!("Result: {}", result);
// }
```

Pinning and Self-Referential Types

Pinning and Self-Referential Types

`Pin` guarantees that an underlying value cannot be moved in memory. This is essential for state machines generated by `async/await` which contain self-referential pointers to local variables on their stack frame.

Async Streams and Error Handling

Async Streams and Error Handling

Streams are asynchronous iterators that yield values over time:

```
// Stream represents a stream of async items
// while let Some(item) = stream.next().await { ... }
```

Building Production Async Applications

Building Production Async Applications

Best practices for async architectures: - Never block async worker threads with long CPU-bound tasks (use `tokio::task::spawn_blocking`). - Always implement timeouts on network requests (`tokio::time::timeout`). - Implement graceful shutdown using cancellation tokens.

Tokio and Rayon: Separate Lanes for I/O and CPU Work

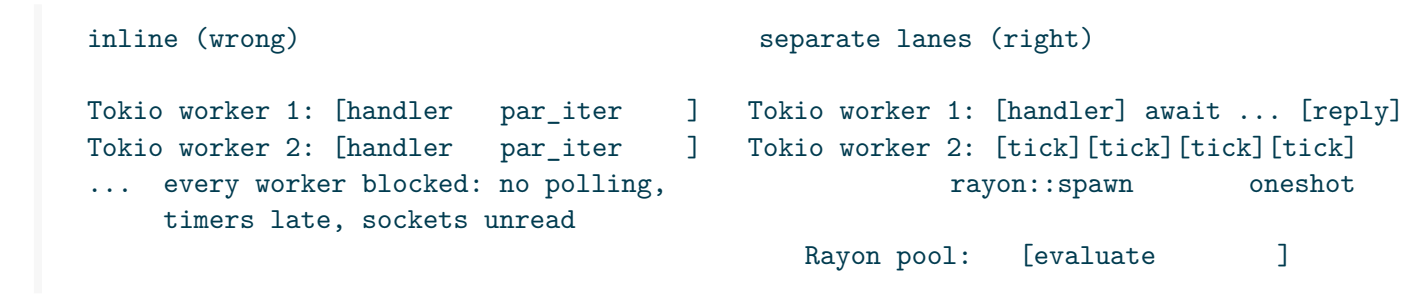
Tokio and Rayon: Separate Lanes for I/O and CPU Work

Thesis

Tokio and Rayon are both thread pools, and both size themselves to "one thread per CPU" by default. They do opposite jobs. Tokio's workers **wait** for many things at once and must come back to the scheduler quickly, at every `.await`. Rayon's workers **compute** and are expected to stay busy until the job is done. Call `par_iter()` from inside an async handler and a Tokio worker stops polling futures until Rayon finishes. Do that on enough workers at the same time and the whole service freezes. Timers fire late, health checks time out, and connection pools look exhausted even though the database is idle.

The fix is not to avoid either library. Give each its own lane: async code hands CPU work to Rayon with `rayon::spawn`, awaits the result on a `tokio::sync::oneshot` channel, and limits how many batches are in flight with a `Semaphore`. This chapter measures the difference with a heartbeat timer, shows how the thread budget changes under a CPU limit, and covers the panic case that turns a bad request into a dead process.

Mental model



Fact	Consequence
A Tokio worker runs one task until it returns `Poll::Pending`	Synchronous work between two `.await`s blocks every other task queued on that worker
`par_iter()` called outside the Rayon pool **blocks the calling thread** until the work is done	Inline Rayon in a handler is blocking code, however parallel it looks

Fact	Consequence
`rayon::spawn` queues a job and returns immediately	The handler can `.await` a channel and give its worker back to Tokio
Both pools default to `available_parallelism()` threads	Two default pools mean 2× oversubscription. `available_parallelism` honours CPU affinity and cgroup quotas, so it shrinks under a container CPU limit
Rayon's queue has no limit	Without a semaphore, a traffic burst becomes an ever-growing queue of CPU work

Worked examples

Tested with rustc 1.99.0, tokio 1.53.2 and rayon 1.12.0 on an 8-CPU Linux machine. Release builds throughout.

```
# Cargo.toml (dependencies)
[dependencies]
rayon = "1.12.0"
tokio = { version = "1.53.2", features = ["rt-multi-thread", "macros", "time", "sync"] }
```

Case 1: Same throughput, very different latency

The program runs 64 CPU-heavy "requests" twice: once calling Rayon inline in the handler and once offloaded. Meanwhile a task ticks every 5 ms, standing in for everything else the service must keep doing, and reports the worst lateness of any tick.

```
// src/main.rs - CPU work inside Tokio: blocking the workers vs. a separate Rayon lane
use rayon::prelude::*;
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::Arc;
use std::time::{Duration, Instant};
use tokio::sync::{oneshot, Semaphore};

/// Stand-in for a CPU-heavy request: pure computation, parallelised with Rayon.
fn evaluate(batch: &[u64]) -> u64 {
    batch
        .par_iter()
        .map(|&x| (0..20_000u64).fold(x, |a, i| a.wrapping_mul(6364136223846793005).wrapping
        .sum()
}

/// Wrong: call Rayon directly on a Tokio worker thread.
async fn handle_inline(batch: Arc<Vec<u64>>) -> u64 {
    evaluate(&batch)
}
```

```
/// Right: bound in-flight batches, hand the work to Rayon, await a oneshot.
async fn handle_offloaded(batch: Arc<Vec<u64>>, permits: Arc<Semaphore>) -> u64 {
    let _permit = permits.acquire_owned().await.expect("semaphore closed");
    let (tx, rx) = oneshot::channel();
    rayon::spawn(move || {
        let _ = tx.send(evaluate(&batch));
    });
    rx.await.expect("rayon job dropped its sender")
}

/// A 5 ms ticker standing in for health checks and socket polling.
/// Returns the worst lateness of any tick until `stop` is set.
async fn ticker(stop: Arc<AtomicBool>) -> Duration {
    let mut worst = Duration::ZERO;
    let mut next = tokio::time::Instant::now();
    while !stop.load(Ordering::Relaxed) {
        next += Duration::from_millis(5);
        tokio::time::sleep_until(next).await;
        worst = worst.max(tokio::time::Instant::now() - next);
    }
    worst
}

#[tokio::main]
async fn main() {
    let batch = Arc::new((0..2_000u64).collect::<Vec<_>>());
    let requests = 64;
    println!(
        "available_parallelism: {}, tokio workers: {}, rayon threads: {}",
        std::thread::available_parallelism().unwrap(),
        tokio::runtime::Handle::current().metrics().num_workers(),
        rayon::current_num_threads()
    );

    for mode in ["inline", "offloaded"] {
        let stop = Arc::new(AtomicBool::new(false));
        let tick = tokio::spawn(ticker(stop.clone()));
        let permits = Arc::new(Semaphore::new(2));
        let start = Instant::now();

        let mut set = tokio::task::JoinSet::new();
        for _ in 0..requests {
            let (b, p) = (batch.clone(), permits.clone());
            set.spawn(async move {
                match mode {
                    "inline" => handle_inline(b).await,
```

```
                    _ => handle_offloaded(b, p).await,
                }
            });
        }
        while set.join_next().await.is_some() {}

        let elapsed = start.elapsed();
        stop.store(true, Ordering::Relaxed);
        let lag = tick.await.unwrap();
        println!(
            "{mode:>9}: {requests} requests in {:>4} ms, worst tick lag {:>3} ms",
            elapsed.as_millis(),
            lag.as_millis()
        );
    }
}
```

```
cargo run --release
```

```
available_parallelism: 8, tokio workers: 8, rayon threads: 8
   inline: 64 requests in   62 ms, worst tick lag  52 ms
offloaded: 64 requests in   61 ms, worst tick lag   2 ms
```

Two more runs gave 55/2 ms and 47/2 ms. Total work time is the same either way, because the same CPUs do the same arithmetic. What changes is whether the runtime stays responsive. Inline, the 5 ms ticker was blocked for most of the batch: in a real service, that is a health check or a socket read waiting 50 ms. Offloaded, it never slipped more than 2 ms.

Case 2: Under a CPU limit

Restrict the process to two CPUs, as a container limit or `taskset` would:

```
taskset -c 0-1 ./target/release/lanes
```

```
available_parallelism: 2, tokio workers: 2, rayon threads: 2
   inline: 64 requests in  210 ms, worst tick lag 204 ms
offloaded: 64 requests in  212 ms, worst tick lag   3 ms
```

With two workers, inline Rayon froze the runtime for the **entire** batch. Both pools shrank to two threads because `available_parallelism()` follows the affinity mask (and on Linux the cgroup CPU quota, too). That is usually what you want, but it is computed from the *limit*, not the *request*. In Kubernetes, a pod with `requests: 2, limits: 8` will start eight threads per pool and get throttled in bursts. Set both pools explicitly:

```
// pools.rs - explicit thread budget for both lanes
fn main() {
    let cpus = std::env::var("CPU_BUDGET").ok().and_then(|v| v.parse().ok()).unwrap_or(2);

    rayon::ThreadPoolBuilder::new()
        .num_threads(cpus)                          // compute lane: all of the budget
        .thread_name(|i| format!("rayon-{i}"))
        .build_global()
        .expect("rayon pool already initialised");

    let rt = tokio::runtime::Builder::new_multi_thread()
        .worker_threads((cpus / 2).max(1))          // I/O lane: mostly parked in .await
        .enable_all()
        .build()
        .unwrap();
    rt.block_on(async { /* serve */ });
}
```

Mild oversubscription (Rayon gets the whole budget, Tokio half) is fine because Tokio's workers spend most of their time parked.

Case 3: Bound the work in flight

`handle_offloaded` takes a permit from a `Semaphore::new(2)` before it spawns. Rayon itself never refuses work, so the semaphore is the only thing that turns a burst into **async waiting** (cheap, cancellable, visible as a queue length you can measure and time out) instead of a growing pile of CPU jobs. Add a timeout around the whole handler to shed load:

```
match tokio::time::timeout(Duration::from_millis(250), handle_offloaded(b, p)).await {
    Ok(v) => Ok(v),
    Err(_) => Err("busy, try again"), // the permit was never taken or is released on drop
}
```

Small batches should skip Rayon entirely. Below a few hundred items, the cost of splitting and joining is larger than the work. Measure the threshold for your own data.

The trap

A panic in a `rayon::spawn` job is not your request's error. With `tokio::spawn`, a panic becomes a `JoinError` for that one task. With `rayon::spawn`, the default is that Rayon **aborts the process**. Run this without a panic handler and the output ends with `Rayon: detected unexpected panic; aborting`. In a server, that kills every in-flight request.

```
// examples/panic.rs - what a panic inside rayon::spawn looks like from the async side
use tokio::sync::oneshot;
```

```rust
#[tokio::main]
async fn main() {
    // Without a panic handler, a panic in rayon::spawn aborts the whole process.
    // Production pools install one; this is the usual "log and carry on" handler.
    rayon::ThreadPoolBuilder::new()
        .panic_handler(|_| eprintln!("rayon: job panicked"))
        .build_global()
        .unwrap();

    let (tx, rx) = oneshot::channel::<u64>();
    rayon::spawn(move || {
        let batch: Vec<u64> = Vec::new();
        let first = batch[0]; // bad input: index out of bounds
        let _ = tx.send(first);
    });

    match rx.await {
        Ok(v) => println!("result: {v}"),
        Err(e) => println!("request failed: {e}"),
    }
}
```

```
cargo run --release --example panic
```

```
thread '<unnamed>' (379384) panicked at examples/panic.rs:16:26:
index out of bounds: the len is 0 but the index is 0
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
rayon: job panicked
request failed: channel closed
```

With a `panic_handler` installed, the process survives. The sender is dropped while the job unwinds, so the handler sees `RecvError` ("channel closed") and can return a 500 for that one request. The handler must not use `.expect()` on the receiver, as Case 1's demo code does, because that just moves the panic into the Tokio task. The alternative is to catch the panic inside the job (`std::panic::catch_unwind`) and send a `Result` over the channel.

The boring rule

> Async handlers never call Rayon (or any long CPU loop) directly. They take a semaphore permit, `rayon::spawn` the work, and await a oneshot whose closed state is an error, not a panic. Both pools get explicit thread counts from the CPU *request*, Rayon has a panic handler, and small batches stay sequential.

Try this

1. Add a sequential threshold to `handle_offloaded` (`if batch.len() < 200 { return evaluate_seq(&batch) }`) and measure tick lag with 20-item batches. Is it acceptable to run that much work inline?
2. Replace `rayon::spawn` with `tokio::task::spawn_blocking(move || evaluate(&batch))`. Compare tick lag and total time, and count threads with `ls /proc/$(pidof lanes)/task | wc -l` during the run.
3. Set the semaphore to 64 permits and then to 1. Record total time and tick lag for each, and explain why neither number moves much while per-request latency does.
4. Wrap the job body in `catch_unwind` and send `Result<u64, String>` so the handler can report *which* input failed.

Sources

- Alice Ryhl, Async: What is blocking? (Tokio maintainer; the `rayon::spawn` + oneshot pattern)
- Tokio docs: `tokio::task` — blocking and yielding, `runtime::Builder::worker_threads`, `sync::Semaphore`, `sync::oneshot`
- Rayon docs: `rayon::spawn` (panic behaviour), `ThreadPoolBuilder::panic_handler`, `num_threads`
- Rust std: `std::thread::available_parallelism` (affinity and cgroup quotas)

Summary

Summary

Chapter Key Takeaways

- Futures are poll-driven state machines executed by a runtime.
- Tokio provides a work-stealing multithreaded async runtime.
- `Pin` prevents memory moves of self-referential generator frames.
- Async code avoids thread exhaustion on high-concurrency I/O workloads.

Macros, Unsafe Rust, and FFI

Declarative Macros with macro_rules!

Declarative Macros with macro_rules!

Declarative macros match against token patterns and expand into Rust code at compile time:

```
macro_rules! make_vec {
    ( $( $x:expr ),* ) => {
        {
            let mut temp_vec = Vec::new();
            $( temp_vec.push($x); )*
            temp_vec
        }
    };
}

fn main() {
    let v = make_vec![1, 2, 3];
    println!("Macro created vec: {:?}", v);
}
```

Procedural Macros and Derive

Procedural Macros and Derive

Procedural macros operate on Rust token streams directly as AST transformations: 1. **Custom Derive:** `#[derive(MyTrait)]` 2. **Attribute-like:** `#[route(GET, "/")]` 3. **Function-like:** `sql!("SELECT * FROM users")`

Understanding unsafe Rust

Understanding unsafe Rust

`unsafe` Rust does not disable the borrow checker; it provides access to five superpowers: 1. Dereferencing raw pointers (`*const T`, `*mut T`). 2. Calling unsafe functions or foreign function interfaces (FFI). 3. Implementing unsafe traits. 4. Mutating mutable static state. 5. Accessing fields of `union` types.

```
fn main() {
    let mut num = 5;
    let r1 = &num as *const i32;

    unsafe {
        println!("Raw pointer points to: {}", *r1);
    }
}
```

Memory Layout and Representation

Memory Layout and Representation

Use `#[repr(C)]` when creating structs whose memory layout must match standard C ABI layouts for FFI or hardware MMIO:

```
#[repr(C)]
struct HardwareRegister {
    control: u32,
    status: u32,
}
```

Foreign Function Interfaces

Foreign Function Interfaces

Call external C libraries safely by wrapping unsafe foreign declarations in safe Rust abstractions:

```
extern "C" {
    fn abs(input: i32) -> i32;
}

fn safe_abs(val: i32) -> i32 {
    unsafe { abs(val) }
}

fn main() {
    println!("Absolute value: {}", safe_abs(-42));
}
```

C-Variadic Functions and VaList

C-Variadic Functions and `VaList`

Thesis

Since **Rust 1.99** (October 2026) you can *define* functions with a C variable argument list — `unsafe extern "C" fn f(fmt: *const c_char, args: ...)` — not just call foreign ones like `printf`. The `...` parameter arrives as a `core::ffi::VaList`, which is layout- and ABI-compatible with the platform's C `va_list`, so you can read arguments yourself or hand the whole list to a C `v*` function such as `vsnprintf`.

That closes a long-standing FFI gap: a Rust library can now implement a C header that contains `void log(int level, const char *fmt, ...)`, sentinel-terminated APIs in the style of `execl`, or plugin callback tables that expect variadic slots — without a C shim file.

The feature is deliberately sharp. A variadic list carries **no count and no types**. Every read is an `unsafe` promise that the caller sent exactly what you are about to take. The job of this chapter is to make that promise small, explicit, and checked by the C compiler wherever possible.

Mental model

```
C caller                          Rust callee (unsafe extern "C" fn f(fixed, ap: ...))

f(2, "x", 3.5f)                   ap: VaList          ← va_start happens for you
      default argument
      promotions:                         ap.next_arg::<*const c_char>()   ← va_arg
       char/short → int                   ap.next_arg::<c_double>()        ← va_arg
       float      → double
                                          ap.clone()                       ← va_copy
registers / stack per ABI
                                           drop(ap)                         ← va_end
```

C	Rust 1.99
`void f(int n, ...)` (definition)	`unsafe extern "C" fn f(n: c_int, mut ap: ...)`
`va_list ap;`	implicit — `ap` *is* the started `VaList`
`va_start(ap, n);`	
`va_arg(ap, int)`	`unsafe { ap.next_arg::<c_int>() }`
`va_copy(dst, src)`	`let dst = src.clone();`
`va_end(ap)`	`Drop` of the `VaList`

C	Rust 1.99
`void vf(int n, va_list ap)`	`unsafe fn vf(n: c_int, ap: VaList<'_>)`

Rules the compiler enforces:

1. **The definition must be `unsafe`.** Callers cannot be checked, so the function is unsafe to call.
2. **ABI must be `"C"` or `"C-unwind"`** for a Rust-bodied definition (other ABIs are only allowed for declarations in `extern` blocks, or naked functions written in assembly).
3. **The `...` parameter needs a name** in a definition (`args: ...`), and it must be last.
4. **Only promotion-safe types can be read.** `next_arg::<T>()` requires the (nightly-only) trait `VaArgSafe`; on stable you just use the concrete types it is implemented for: `c_int`/`c_uint` and wider integers, `c_double`, and raw pointers. `f32`, `i8`, `u16`, and `bool` are rejected because C never passes them through `...` unpromoted.

Rules the compiler **cannot** enforce: how many arguments there are, and what type each one has. Those come from a *protocol* — a count, a format string, or a sentinel — that you document in the `# Safety` section and, ideally, encode in the C header.

Worked examples

All listings were built with `rustc 1.99.0 (b940084d7 2026-09-28)` and GCC 14.2 on x86_64 Linux, edition 2024.

Case 1: count-prefixed list, defined and called in Rust

The simplest protocol: the first fixed argument says how many follow. The second function shows the v-variant split you will want for anything non-trivial — a thin variadic entry point that forwards its `VaList` to an ordinary `unsafe fn`.

```
// sum.rs - define and call a C-variadic function entirely in Rust.
use std::ffi::{c_double, c_int, VaList};

/// # Safety
/// The caller must pass exactly `count` further arguments, each a C `int`.
unsafe extern "C" fn sum_ints(count: c_int, mut args: ...) -> c_int {
    let mut total = 0;
    for _ in 0..count {
        // SAFETY: the caller promised `count` ints follow.
        total += unsafe { args.next_arg::<c_int>() };
    }
    total
}
```

```rust
/// The `v`-variant: takes an already-started list, like C's `vprintf`.
///
/// # Safety
/// `ap` must hold exactly `count` C `double`s.
unsafe fn vmean(count: c_int, mut ap: VaList<'_>) -> c_double {
    let mut acc = 0.0;
    for _ in 0..count {
        acc += unsafe { ap.next_arg::<c_double>() };
    }
    acc / count as c_double
}

/// # Safety
/// The caller must pass exactly `count` further `double`s.
unsafe extern "C" fn mean(count: c_int, args: ...) -> c_double {
    unsafe { vmean(count, args) }
}

fn main() {
    let s = unsafe { sum_ints(3, 10, 20, 12) };
    let m = unsafe { mean(4, 1.0, 2.0, 3.0, 4.5) };
    let small: u8 = 7;
    let k = unsafe { sum_ints(2, small as c_int, -2) }; // explicit promotion
    println!("sum = {s}, mean = {m}, k = {k}");
}
```

```
rustc --edition 2024 sum.rs && ./sum
```

```
sum = 42, mean = 2.625, k = 5
```

Note `small as c_int`: when *Rust* calls a variadic function, it refuses to pass `u8` or `f32` and asks you to promote by hand (see *The trap*).

Case 2: a printf-style logger implemented in Rust, called from C

This is the case the feature exists for. A C program (or a C plugin host) expects `rlog(int level, const char *fmt, ...)`. The Rust implementation never parses the format string itself; it forwards the list to libc's `vsnprintf`, which is the only component that actually understands `%s`, `%.1f`, and `%llu`.

Two details matter:

- `vsnprintf` **consumes** the `va_list` it receives. To measure first and then render, pass `ap.clone()` (C `va_copy`) to the measuring call and the original to the rendering call.
- `VaList` can be passed **by value across FFI** — declare the C parameter as `ap: VaList` in the `extern` block.

```rust
// rlog.rs - printf-style logging and a sentinel-terminated API,
// implemented in Rust and exported with the C ABI.
use std::ffi::{c_char, c_int, CStr, VaList};
use std::ptr;

unsafe extern "C" {
    // int vsnprintf(char *str, size_t size, const char *format, va_list ap);
    fn vsnprintf(buf: *mut c_char, size: usize, fmt: *const c_char, ap: VaList) -> c_int;
}

/// Format `fmt` with `ap` into an owned Rust `String`.
///
/// # Safety
/// `fmt` must be a valid NUL-terminated C format string and `ap` must hold
/// arguments matching it exactly (same rules as C `vprintf`).
unsafe fn vformat(fmt: *const c_char, ap: VaList<'_>) -> Option<String> {
    // Pass 1: measure. vsnprintf consumes the list it is given, so hand it a
    // clone (C: va_copy) and keep the original for pass 2.
    let needed = unsafe { vsnprintf(ptr::null_mut(), 0, fmt, ap.clone()) };
    if needed < 0 {
        return None;
    }
    // Pass 2: render into a buffer with room for the trailing NUL.
    let mut buf = vec![0u8; needed as usize + 1];
    let written = unsafe { vsnprintf(buf.as_mut_ptr().cast(), buf.len(), fmt, ap) };
    if written != needed {
        return None;
    }
    buf.truncate(needed as usize);
    String::from_utf8(buf).ok()
}

/// C signature: `void rlog(int level, const char *fmt, ...);`
///
/// # Safety
/// Same contract as `printf`: the variadic arguments must match `fmt`.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rlog(level: c_int, fmt: *const c_char, args: ...) {
    let tag = match level {
        0 => "DEBUG",
        1 => "INFO",
        2 => "WARN",
        _ => "ERROR",
    };
    match unsafe { vformat(fmt, args) } {
        Some(msg) => eprintln!("[{tag}] {msg}"),
        None => {
```

```
            let raw = unsafe { CStr::from_ptr(fmt) };
            eprintln!("[{tag}] <format error: {raw:?}>");
        }
    }
}

/// C signature: `size_t rjoin_len(const char *first, ...);`
/// Sum of strlen() of every string up to a terminating NULL pointer.
///
/// # Safety
/// Every argument must be a `const char *` to a NUL-terminated string,
/// and the list must end with `(char *)NULL`.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rjoin_len(first: *const c_char, mut rest: ...) -> usize {
    let mut total = 0;
    let mut cur = first;
    while !cur.is_null() {
        total += unsafe { CStr::from_ptr(cur) }.count_bytes();
        cur = unsafe { rest.next_arg::<*const c_char>() };
    }
    total
}
```

The public C header is where you buy back type checking. `format(printf, 2, 3)` tells GCC and Clang that argument 2 is a printf format and checking starts at argument 3; `sentinel` demands a trailing null pointer.

```
/* rlog.h - public C header for the Rust-implemented logger. */
#ifndef RLOG_H
#define RLOG_H
#include <stddef.h>

/* format(printf, 2, 3): argument 2 is the format, checking starts at 3. */
void rlog(int level, const char *fmt, ...) __attribute__((format(printf, 2, 3)));

/* sentinel: the variadic list must end with a NULL pointer. */
size_t rjoin_len(const char *first, ...) __attribute__((sentinel));

#endif
```

```
/* main.c - C caller for the Rust-implemented variadic functions. */
#include <stdio.h>
#include "rlog.h"

int main(void) {
    rlog(1, "listening on %s:%d", "0.0.0.0", 8443);
```

```
    rlog(2, "disk %s at %.1f%% (%llu bytes free)", "/var", 91.5, 4294967296ULL);
    rlog(3, "%s", "plain string, no surprises");

    size_t n = rjoin_len("usr", "local", "bin", (char *)NULL);
    printf("rjoin_len = %zu\n", n);
    return 0;
}
```

Build the Rust side as a static library, then link it into the C program with the C compiler driver:

```
rustc --edition 2024 -O --crate-type=staticlib rlog.rs -o librlog.a
cc -std=c17 -Wall -Wextra main.c -L. -lrlog -o demo
./demo
```

```
[INFO] listening on 0.0.0.0:8443
[WARN] disk /var at 91.5% (4294967296 bytes free)
[ERROR] plain string, no surprises
rjoin_len = 11
```

If the link fails with undefined symbols from libc, pthreads, or libdl, ask `rustc` what the static library needs and append it to the `cc` line:

```
rustc --edition 2024 --crate-type=staticlib rlog.rs -o librlog.a --print native-static-libs
```

```
note: native-static-libs: -lgcc_s -lutil -lrt -lpthread -lm -ldl -lc
```

In a Cargo project the equivalent is `[lib] crate-type = ["staticlib"]` (or `"cdylib"`) in `Cargo.toml`; nothing else about the code changes.

Case 3: the header catches caller mistakes before they become UB

The Rust callee cannot detect a wrong argument type — it will happily read 4 bytes of an 8-byte `long long`. The defence lives at the call site, in the C compiler, driven by the attributes in `rlog.h`:

```
/* bad.c - mistakes the header attributes catch at compile time. */
#include "rlog.h"

int main(void) {
    long long bytes = 4294967296LL;
    rlog(1, "free: %d bytes", bytes);        /* wrong conversion */
    return (int)rjoin_len("usr", "bin");     /* missing NULL sentinel */
}
```

```
cc -std=c17 -Wall -Wextra -c bad.c -o /dev/null
```

```
bad.c: In function ‘main’:
bad.c:6:21: warning: format ‘%d’ expects argument of type ‘int’, but argument 3 has type ‘lo
    6 |     rlog(1, "free: %d bytes", bytes);        /* wrong conversion */
      |                     ~^         ~~~~~
      |                      |         |
      |                      int       long long int
      |                     %lld
bad.c:7:5: warning: missing sentinel in function call [-Wformat=]
    7 |     return (int)rjoin_len("usr", "bin");     /* missing NULL sentinel */
      |     ^~~~~~
```

Clang reports the same two problems (`-Wformat`, `-Wsentinel`) and even suggests the fix (`, NULL`). In CI, build C callers with `-Werror=format` (GCC files the missing-sentinel warning under `-Wformat`; Clang under `-Wsentinel`, so add `-Werror=sentinel` there) so these never ship.

Case 4: what the compiler refuses

Three mistakes are rejected at compile time on Rust 1.99. Seeing the errors once makes the rules stick.

A definition that is not `unsafe`:

```
// not_unsafe.rs
use std::ffi::c_int;
extern "C" fn count_args(_n: c_int, _args: ...) {}
fn main() {}
```

```
rustc --edition 2024 not_unsafe.rs
```

```
error: functions with a C variable argument list must be unsafe
 --> not_unsafe.rs:3:37
  |
3 | extern "C" fn count_args(_n: c_int, _args: ...) {}
  |                                     ^^^^^^^^^^
  |
help: add the `unsafe` keyword to this definition
  |
3 | unsafe extern "C" fn count_args(_n: c_int, _args: ...) {}
  | ++++++

error: aborting due to 1 previous error
```

Reading a type that C always promotes:

```
// read_f32.rs
unsafe extern "C" fn first_float(mut args: ...) -> f32 {
```

```
    unsafe { args.next_arg::<f32>() }
}
fn main() {}
```

```
rustc --edition 2024 read_f32.rs
```

```
error[E0277]: the trait bound `f32: VaArgSafe` is not satisfied
 --> read_f32.rs:3:30
  |
3 |     unsafe { args.next_arg::<f32>() }
  |                   --------   ^^^ the nightly-only, unstable trait `VaArgSafe` is not imp
  |                   |
  |                   required by a bound introduced by this call
  |
  = help: the following other types implement trait `VaArgSafe`:
            f64
            i128
            i32
            i64
            isize
            u128
            u32
            u64
            usize
note: required by a bound in `VaList::<'f>::next_arg`
```

(`i16`, `u8`, and `bool` fail the same way. Read `c_int` / `c_uint` / `c_double` and narrow afterwards with a checked conversion.)

Passing an unpromoted type *from Rust*:

```
// promote.rs
use std::ffi::c_int;

unsafe extern "C" fn sum_ints(count: c_int, mut args: ...) -> c_int {
    let mut total = 0;
    for _ in 0..count {
        total += unsafe { args.next_arg::<c_int>() };
    }
    total
}

fn main() {
    let small: u8 = 7;
    let ratio: f32 = 0.5;
    let s = unsafe { sum_ints(2, small, ratio) };
```

```
    println!("{s}");
}
```

```
rustc --edition 2024 promote.rs
```

```
error[E0617]: can't pass `u8` to variadic function
  --> promote.rs:15:34
   |
15 |     let s = unsafe { sum_ints(2, small, ratio) };
   |                                  ^^^^^
   |
help: cast the value to `c_uint`
   |
15 |     let s = unsafe { sum_ints(2, small as c_uint, ratio) };
   |                                         +++++++++

error[E0617]: can't pass `f32` to variadic function
  --> promote.rs:15:41
   |
15 |     let s = unsafe { sum_ints(2, small, ratio) };
   |                                          ^^^^^
   |
help: cast the value to `c_double`
   |
15 |     let s = unsafe { sum_ints(2, small, ratio as c_double) };
   |                                                ++++++++++++

error: aborting due to 2 previous errors
```

Notice the second error also exposes a *logic* bug the compiler cannot see: `sum_ints` reads `c_int`s, but the caller is now about to send a `c_double`. Casting to silence E0617 is correct only if the cast type matches what the callee reads.

The trap

Treating `next_arg` as if it were type-checked. It is a raw read guided only by your protocol. Every one of these is undefined behaviour, and none produces a compile error in the callee:

- **Reading past the end.** There is no length; `next_arg` on an exhausted list reads whatever is in the next register or stack slot. A missing sentinel in Case 2 walks off into garbage pointers.
- **Reading the wrong type.** Caller passes `long long`, you read `c_int`; caller passes `double`, you read a pointer. The `VaList` docs allow only *compatible* types: identical types, same-size integers whose value fits both, compatible pointers, or `*const c_void` ↔ `*const u8`/`i8`.
- **Trusting integer widths across platforms.** `c_long` is 64-bit on Linux x86_64 and 32-bit on Windows x86_64. Read `c_long` when the C side passes `long`, not `i64`. The `VaArgSafe` docs say implementations for `i32`/`usize` "shouldn't be relied upon directly"; spell the C type.

- **Re-reading a consumed list.** After `vsnprintf(…, ap)` the list is spent; use `ap.clone()` *before* the first consumer if you need two passes.
- **Panicking out of an `extern "C"` function.** A panic that reaches an `extern "C"` boundary aborts the process. If unwinding through C is genuinely intended, use `"C-unwind"`; otherwise catch and convert errors to return codes, as `rlog` does with its `None` branch.
- **Calling your own variadic from Rust.** Rust callers get none of the C header's format checking. Give Rust code a normal, safe function (`fn log(level: Level, msg: &str)`) and keep the variadic entry point for C.

The boring rule

> A C-variadic function in Rust is a thin, `unsafe`, `#[unsafe(no_mangle)]` door for C callers. It reads arguments only under a documented protocol — a count, a format string handed to libc's `v*` functions, or a NULL sentinel — reads only C-spelled promoted types (`c_int`, `c_uint`, `c_long`, `c_double`, pointers), clones before any second pass, never unwinds through `extern "C"`, and ships with a C header carrying `format(printf, …)` or `sentinel` so the *caller's* compiler checks what the callee cannot.

Checklist before merging:

1. `# Safety` section states the exact protocol (how many, which types, how the list ends).
2. Real formatting is delegated to `vsnprintf`/`vfprintf`; you do not reimplement `%` parsing.
3. Every read type is a `core::ffi` C type or a raw pointer.
4. The C header has `__attribute__((format(printf, N, M)))` or `__attribute__((sentinel))`, and CI builds C callers with `-Werror=format`.
5. Rust-side callers use a safe non-variadic API instead.

Try this

1. Add `rlog_count(void)` to `rlog.rs` that returns how many messages have been logged (an `AtomicU32`), call it from `main.c`, and confirm it prints `3`.
2. Remove `(char *)NULL` from the `rjoin_len` call in `main.c`, rebuild with `-Wall`, and read the warning. Then delete `__attribute__((sentinel))` from the header and observe that the warning disappears while the bug remains — that is why the attribute is part of the API.
3. Write `vrlog(int level, const char *fmt, va_list ap)` in Rust taking `ap: VaList<'_>`, make `rlog` a one-line forwarder to it, and export both. Call `vrlog` from a C wrapper function that does its own `va_start`/`va_end`.
4. Change `vformat` to pass `ap` (not `ap.clone()`) to the measuring `vsnprintf` call. Explain why the second call is now undefined behaviour even if it appears to work on your machine.
5. Run `rustc --print native-static-libs` for your target and wire the result into a `Makefile` rule so the link line is never guessed.

Sources

- Announcing Rust 1.99.0 — extern "C" variadics (Rust Blog, 2026-10-01)
- `core::ffi::VaList` and `VaArgSafe` — std/core API docs
- The Rust Reference — Functions: C-variadic parameters and External blocks: variadic functions
- GCC manual, *Common Function Attributes* (`format`, `sentinel`)

Summary

Summary

Chapter Key Takeaways

- Declarative macros (`macro_rules!`) perform hygienic pattern-based code generation.
- Procedural macros transform token streams at compile time.
- `unsafe` isolates low-level pointer manipulation and FFI behind safe API boundaries.
- `#[repr(C)]` guarantees binary compatibility with standard C ABIs.

Testing, Documentation, Performance, and Deployment

Unit Tests, Integration Tests, and Doctests

Unit Tests, Integration Tests, and Doctests

Rust has first-class built-in testing support.

Unit Tests

Unit tests live in the same source files as the code they test inside a `#[cfg(test)]` module:

```rust
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_add() {
        assert_eq!(add(2, 3), 5);
    }
}
```

Integration Tests

Integration tests live in the top-level `tests/` directory and test the public API as an external consumer.

Documentation Tests (Doctests)

Examples written in markdown doc-comments (`///`) are compiled and executed during `cargo test`.

Benchmarking with Criterion

Benchmarking with Criterion

Criterion provides robust statistical benchmarking with outlier detection and regression analysis:

```
[dev-dependencies]
criterion = "0.5"

[[bench]]
name = "compute_benchmark"
harness = false
```

Writing Great Documentation

Writing Great Documentation

Write documentation comments using triple slashes `///` supporting Markdown:

```
/// Calculates the checksum for a given network payload buffer.
///
/// # Examples
///
/// ```
/// let data = [1, 2, 3];
/// let sum: u32 = data.iter().map(|&x| x as u32).sum();
/// assert_eq!(sum, 6);
/// ```
pub fn calculate_checksum(data: &[u8]) -> u32 {
    data.iter().map(|&x| x as u32).sum()
}
```

Generate and open documentation:

```
cargo doc --open
```

Profiling and Optimization Techniques

Profiling and Optimization Techniques

Techniques for production profiling: - Use `perf` or `samply` on Linux to capture CPU flamegraphs. - Configure Link-Time Optimization (LTO) and codegen units in `Cargo.toml`:

```
[profile.release]
opt-level = 3
lto = "fat"
codegen-units = 1
panic = "abort"
strip = true
```

Building and Deploying Rust Applications

Building and Deploying Rust Applications

Rust binaries can be statically compiled with musl libc for minimal, scratch Docker containers:

```
rustup target add x86_64-unknown-linux-musl
cargo build --release --target x86_64-unknown-linux-musl
```

Summary

Summary

Chapter Key Takeaways

- Unit tests, integration tests, and doctests are integrated into `cargo test`.
- Criterion provides statistical benchmarking.
- Triple-slash doc comments generate standard HTML documentation.
- Release profiles with LTO produce compact, high-performance static binaries.

The Rust Ecosystem: Essential Crates for Production Development

serde: Serialization and Deserialization

serde: Serialization and Deserialization

Serde is the framework for serializing and deserializing Rust data structures efficiently and generically.

```
// serde derives Serialize and Deserialize for zero-allocation JSON parsing
// #[derive(Serialize, Deserialize, Debug)]
// struct User { id: u64, username: String }
```

clap: Command-Line Argument Parsing

clap: Command-Line Argument Parsing

Clap provides declarative, type-safe command-line argument parsing via procedural derive macros:

```
// #[derive(Parser)]
// #[command(author, version, about)]
// struct Cli { port: u16 }
```

reqwest: HTTP Client

reqwest: HTTP Client

Reqwest provides ergonomic asynchronous and synchronous HTTP clients with connection pooling, TLS, and JSON decoding.

axum: Web Framework

axum: Web Framework

Axum is a modular, ergonomic web application framework built on top of Tokio, Tower, and Hyper.

actix-web: High-Performance Web Framework

actix-web: High-Performance Web Framework

Actix-web is a high-performance, actor-inspired web framework frequently ranking among the fastest web servers in industry benchmarks.

Project 2: Async HTTP API with axum, sqlx, and tracing

Project 2: Async HTTP API with axum, sqlx, and tracing

In Project 2, we build an asynchronous microservice handling user registrations and order audits backed by SQLite/PostgreSQL with structured logging:

```
// Architecture of the User Service API
// Router binds routes: POST /users, GET /users/:id
// Handlers extract typed JSON, execute SQL queries via sqlx, and emit tracing spans
```

Testing the User API

Testing the User API

End-to-end integration tests using Tower `ServiceExt::oneshot` to simulate HTTP requests without binding network sockets.

Benchmarking the User API

Benchmarking the User API

Load-testing the async API with `oha` or `wrk` measuring request latencies under sustained concurrent connections.

Deploying the User API

Deploying the User API

Containerizing the API into a multi-stage Docker build producing a single 15MB container running as an unprivileged user.

sqlx: Async Database Access

sqlx: Async Database Access

SQLx provides compile-time verified SQL queries without an ORM, checking query syntax against live database schemas during `cargo build`.

rayon: Data Parallelism

rayon: Data Parallelism

Rayon converts sequential iterator chains into parallel work-stealing pipelines with a simple `.par_iter()` drop-in replacement.

tracing: Structured Logging and Observability

tracing: Structured Logging and Observability

Tracing instruments Rust applications with contextual, structured events and spans compatible with OpenTelemetry and Jaeger.

chrono: Date and Time

chrono: Date and Time

Chrono handles ISO 8601 timestamps, UTC conversions, duration arithmetic, and timezone formatting.

uuid: Universally Unique Identifiers

uuid: Universally Unique Identifiers

Fast, cryptographically secure generation and parsing of UUID v4 and v7 identifiers.

rand: Random Number Generation

rand: Random Number Generation

Generates cryptographically secure pseudorandom numbers and provides distribution sampling.

regex: Regular Expressions

regex: Regular Expressions

Linear-time regular expression matching guaranteeing no catastrophic backtracking.

bytes: Efficient Binary Buffers

bytes: Efficient Binary Buffers

`Bytes` and `BytesMut` provide reference-counted zero-copy slicing for network packet processing.

futures: Async Primitives

futures: Async Primitives

Provides channels, select combinators, and join mechanisms for working with asynchronous streams.

Summary

Summary

Chapter Key Takeaways

- Serde, Clap, Reqwest, Axum, and SQLx form the production backbone for web backends and CLIs.
- SQLx enables compile-time verified database operations.
- Tracing provides production-ready observability.
- The Rust crate ecosystem provides zero-cost modular tools for systems engineering.

Advanced Topics: Compiler, Debugging, Security, and Anti-Patterns

The Rust Compilation Pipeline

The Rust Compilation Pipeline

Understanding how `rustc` compiles code: 1. Parsing & Macro Expansion -> AST 2. HIR (High-level Intermediate Representation) -> Type checking 3. MIR (Mid-level Intermediate Representation) -> Borrow checker & optimizations 4. LLVM IR -> Native machine code generation

Debugging Rust Programs

Debugging Rust Programs

Debugging Rust applications using `gdb`, `lldb`, and Rust backtraces:

```
RUST_BACKTRACE=1 cargo run
```

Security Considerations in Rust

Security Considerations in Rust

- Audit dependencies using `cargo-audit` and `cargo-vet`.
- Avoid integer overflow panics in production using checked/saturating arithmetic.
- Sanitize memory buffers holding sensitive cryptographic keys with crates like `zeroize`.

Common Anti-Patterns and How to Avoid Them

Common Anti-Patterns and How to Avoid Them

- **Anti-pattern:** Excessive `.clone()` calls to appease the borrow checker. *Remedy:* Restructure ownership or pass references.
- **Anti-pattern:** Unnecessary `Arc<Mutex<T>>` when local message passing suffices. *Remedy:* Use channels or scoped threads.
- **Anti-pattern:** Overusing `Box<dyn Error>` in public library crates. *Remedy:* Expose custom structured error enums.

Summary

Summary

Chapter Key Takeaways

- Understanding the AST -> HIR -> MIR -> LLVM pipeline clarifies compiler errors and optimizations.
- `RUST_BACKTRACE=1` and debuggers demystify runtime failures.
- Regular security audits with `cargo-audit` prevent supply-chain vulnerabilities.
- Writing idiomatic Rust eliminates defensive over-cloning and unnecessary synchronization locks.

WebAssembly with Rust

Why Rust for WebAssembly?

Why Rust for WebAssembly?

WebAssembly (Wasm) is a binary instruction format for a stack-based virtual machine designed for near-native execution speed on the web and edge servers.

Rust is uniquely positioned as the premier language for WebAssembly: - **No Garbage Collector:** Wasm has no built-in GC; Rust's compile-time memory management avoids bundling heavy GC runtimes into the `.wasm` binary. - **Compact Binaries:** Tiny output sizes optimize load times across web browsers and serverless functions. - **Rich Tooling:** `wasm-pack` and `wasm-bindgen` provide seamless high-level JavaScript interoperability.

WebAssembly Fundamentals

WebAssembly Fundamentals

Wasm executes inside a sandboxed linear memory space with strict type checking and deterministic execution semantics.

Setting Up: wasm-pack and wasm-bindgen

Setting Up: wasm-pack and wasm-bindgen

Install `wasm-pack`:

```
cargo install wasm-pack
```

Configure `Cargo.toml`:

```
[lib]
crate-type = ["cdylib", "rlib"]

[dependencies]
wasm-bindgen = "0.2"
```

Building Your First WebAssembly Module

Building Your First WebAssembly Module

```
// In src/lib.rs
// use wasm_bindgen::prelude::*;
//
// #[wasm_bindgen]
// pub fn add(a: i32, b: i32) -> i32 {
//     a + b
// }
```

Build the npm package:

```
wasm-pack build --target web
```

Interacting with the DOM

Interacting with the DOM

Using `web-sys` to interact directly with DOM nodes, canvases, and local storage from Rust.

Handling Events and Async

Handling Events and Async

Attaching browser event listeners with closures and running async promises with `wasm-bindgen-futures`.

Performance Considerations

Performance Considerations

Optimizing Wasm size and throughput using `wasm-opt` from binaryen and disabling panic formatting.

Server-Side WebAssembly

Server-Side WebAssembly

Running Wasm outside the browser using WASI (WebAssembly System Interface) and runtimes like Wasmtime, Wasmer, and Cloudflare Workers.

Publishing and Distribution

Publishing and Distribution

Distributing Wasm modules via npm registries or OCI container registries for serverless edge deployments.

Summary

Summary

Chapter Key Takeaways

- Rust's lack of a GC runtime makes it ideal for compact, high-speed WebAssembly modules.
- `wasm-pack` packages Rust code into web-ready JavaScript/Wasm bundles.
- WASI enables secure, sandboxed server-side WebAssembly execution.

Embedded Rust and no_std Development

Understanding no_std

Understanding no_std

Embedded microcontrollers lack operating system services like virtual memory, filesystems, and threads. In embedded Rust, programs declare `#![no_std]`, compiling without the standard library while retaining the core language and primitives.

```
// #![no_std]
// #![no_main]
//
// use core::panic::PanicInfo;
//
// #[panic_handler]
// fn panic(_info: &PanicInfo) -> ! {
//     loop {}
// }
```

Choosing a Target: Cortex-M

Choosing a Target: Cortex-M

Targeting ARM Cortex-M microcontrollers using `rustup target add thumbv7em-none-eabihf`.

Writing an Embedded Application: Blinking an LED

Writing an Embedded Application: Blinking an LED

The classic embedded "Hello World": toggling a GPIO pin tied to an LED using Peripheral Access Crates (PAC) and Hardware Abstraction Layers (HAL).

Memory Management Without a Heap

Memory Management Without a Heap

Managing state using static memory and stack buffers with zero dynamic heap allocation.

Interrupts and Concurrency

Interrupts and Concurrency

Handling hardware interrupt service routines (ISRs) safely using critical sections and interrupt-safe atomics.

Choosing an Executor or RTOS

Choosing an Executor or RTOS

Comparing bare-metal superloops, RTIC (Real-Time Interrupt-driven Concurrency), and Embassy (async embedded executor).

Testing Embedded Code

Testing Embedded Code

Unit testing business logic on host architectures and running on-target tests with `probe-rs`.

Toolchain and Debugging

Toolchain and Debugging

Hardware flashing and in-circuit debugging via SWD/JTAG with `probe-rs` and `gdb`.

Summary

Summary

Chapter Key Takeaways

- `#![no_std]` strips OS dependencies for bare-metal firmware development.
- Hardware Abstraction Layers (HAL) provide safe, type-checked register manipulation.
- Critical sections and atomics guarantee interrupt safety without data corruption.

Conclusion: The Rust Mindset

Core Principles

Core Principles

Looking back over our journey, three fundamental engineering principles define idiomatic Rust:

1. **Safety is an Asset, Not a Constraint:** The borrow checker is an automated proof assistant ensuring thread and memory safety.
2. **Explicit over Implicit:** Errors are values, allocations are intentional, and data flows are visible.
3. **Zero-Cost Abstractions:** High-level expressiveness compiles down to optimal low-level machine instructions.

Where Rust Shines

Where Rust Shines

Rust delivers unmatched value in: - High-performance network proxies, database engines, and storage systems. - Operating system kernels, drivers, and hypervisors. - Low-latency financial trading platforms. - WebAssembly client-side computational modules and edge serverless workers. - Embedded firmware and safety-critical medical and automotive controllers.

Where Rust May Not Be the Best Fit

Where Rust May Not Be the Best Fit

Rust may introduce unnecessary friction for: - Disposable one-off scripts where Python or Bash finishes the job in minutes. - Rapidly churning early-stage UI prototypes where types and domain models change hourly. - Systems where memory footprint and latency do not matter and teams already possess deep expertise in managed languages.

Continuing Your Journey

Continuing Your Journey

To continue deepening your Rust mastery: - Read *The Rust Reference* and *The Rustonomicon* for deep language semantics and unsafe invariants. - Explore open-source projects on crates.io to observe production architectures. - Contribute bug fixes and documentation improvements to the community.

Final Thoughts

Final Thoughts

Rust changes the way you think about programming. Even when writing Go, C, or Python, the habits ingrained by Rust—thinking carefully about ownership, data sharing, mutability, and error handling—will make you a vastly more rigorous systems engineer.

Welcome to the community, and build fearless software.